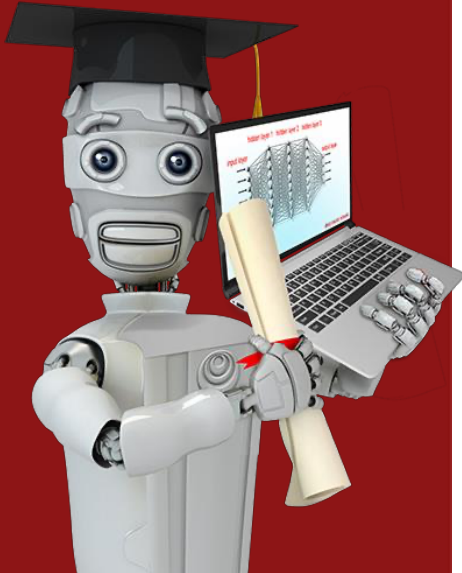




Stanford
ONLINE



Advanced Learning Algorithms

Welcome!

Advanced learning algorithms

Neural Networks 

inference (prediction)

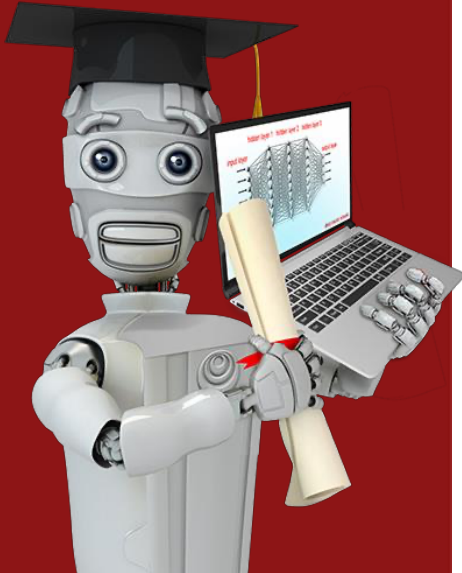
training

Practical advice for building machine learning systems



Decision Trees





Neural Networks Intuition

Neurons and the brain

Neural networks



Origins: Algorithms that try to mimic the brain.

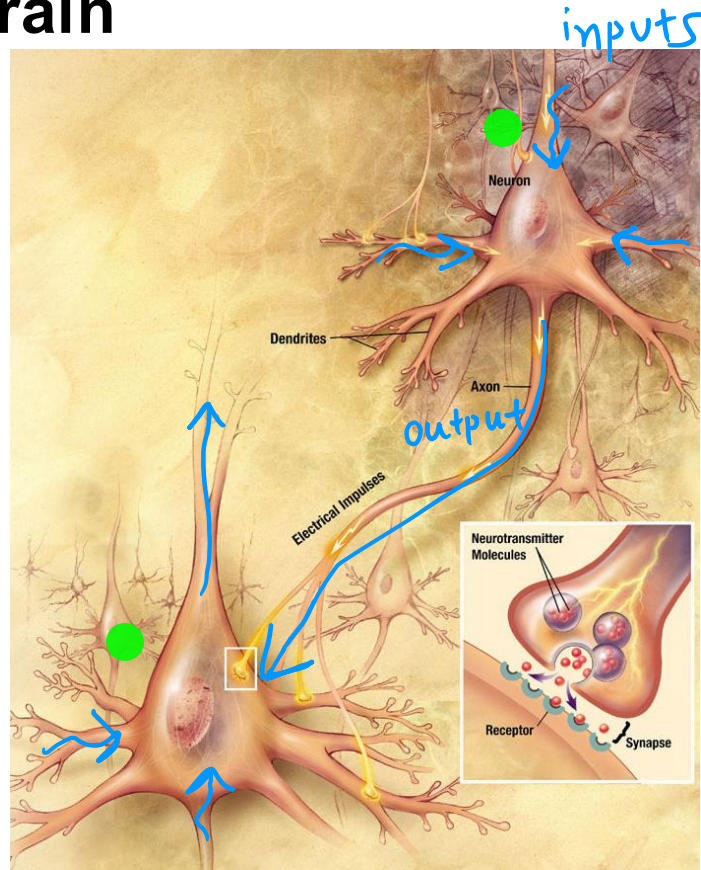
Used in the 1980's and early 1990's.

Fell out of favor in the late 1990's.

Resurgence from around 2005.

speech → images → text (NLP) → ...

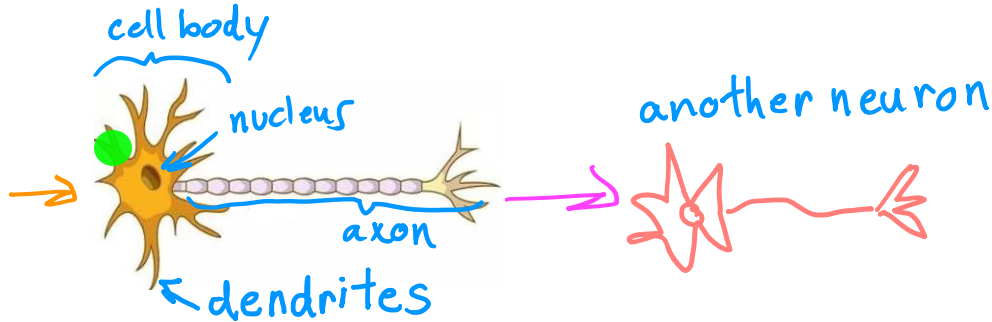
Neurons in the brain



Biological neuron

inputs

outputs



Simplified mathematical model of a neuron

inputs

outputs

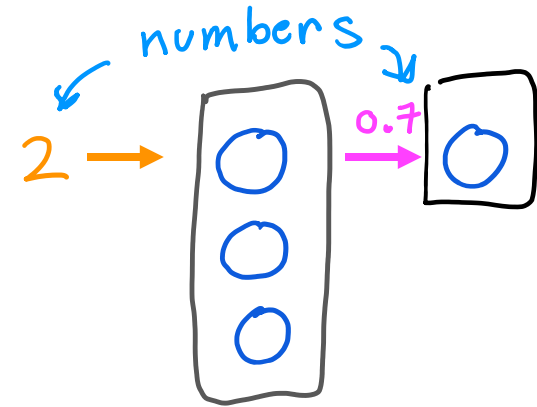
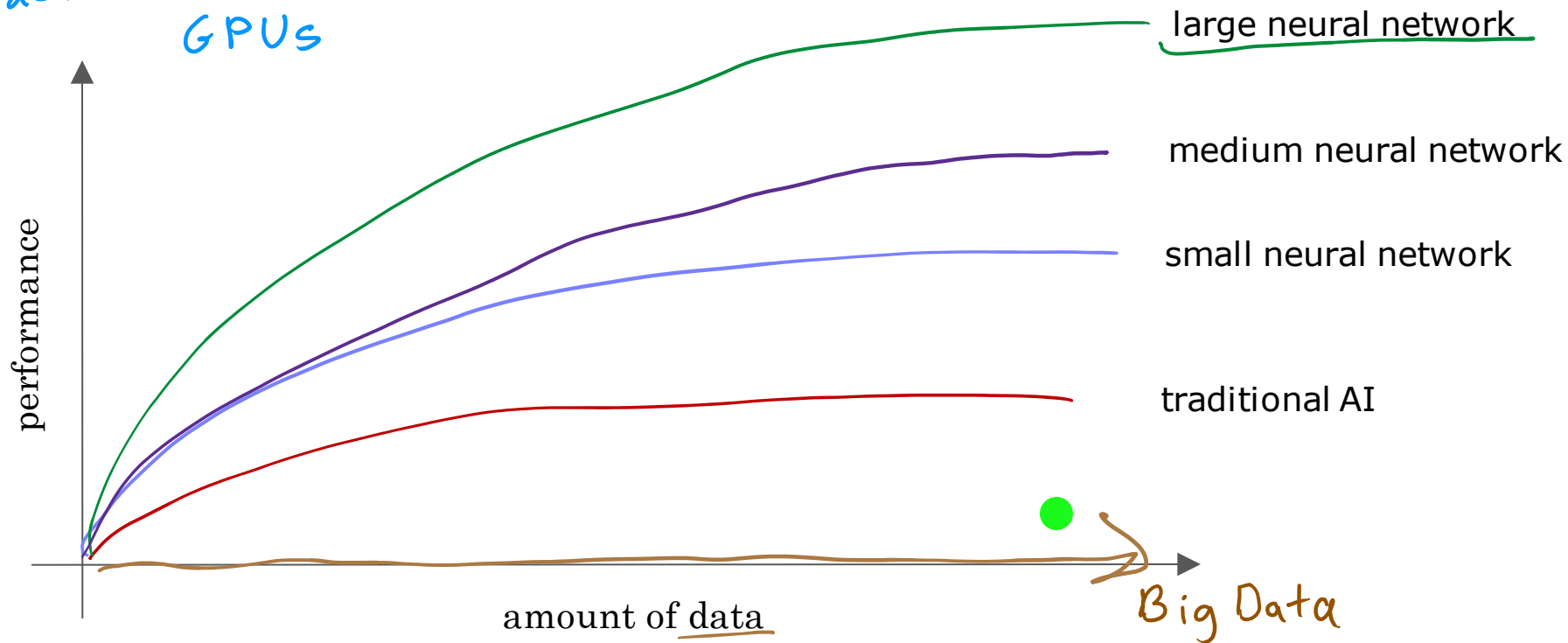


image source: <https://biologydictionary.net/sensory-neuron/>

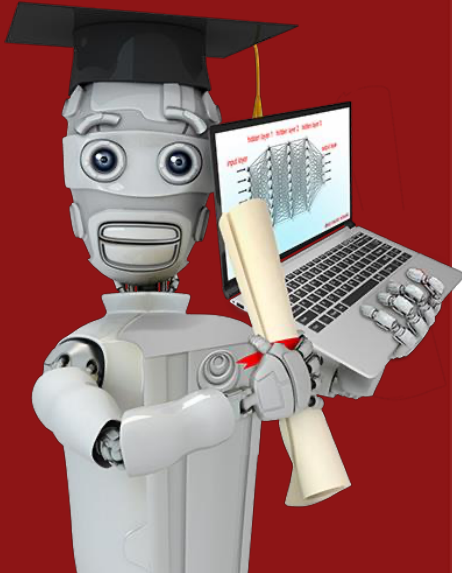
Why Now?

Faster computer processors
GPUs





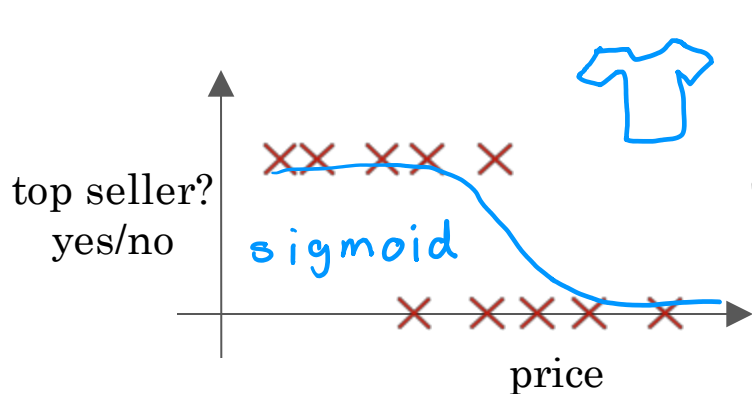
Stanford
ONLINE



Neural Network Intuition

Demand Prediction

Demand Prediction



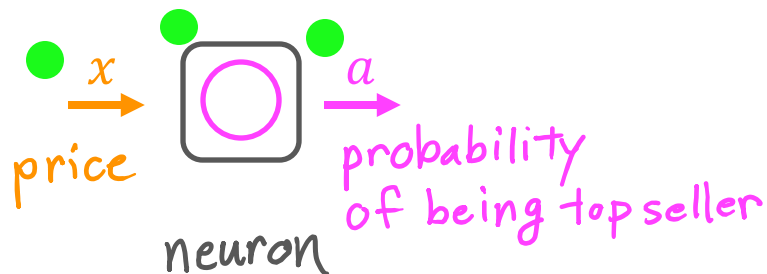
$x = \text{price}$

$a = f(x) = \frac{1}{1 + e^{-(wx+b)}}$

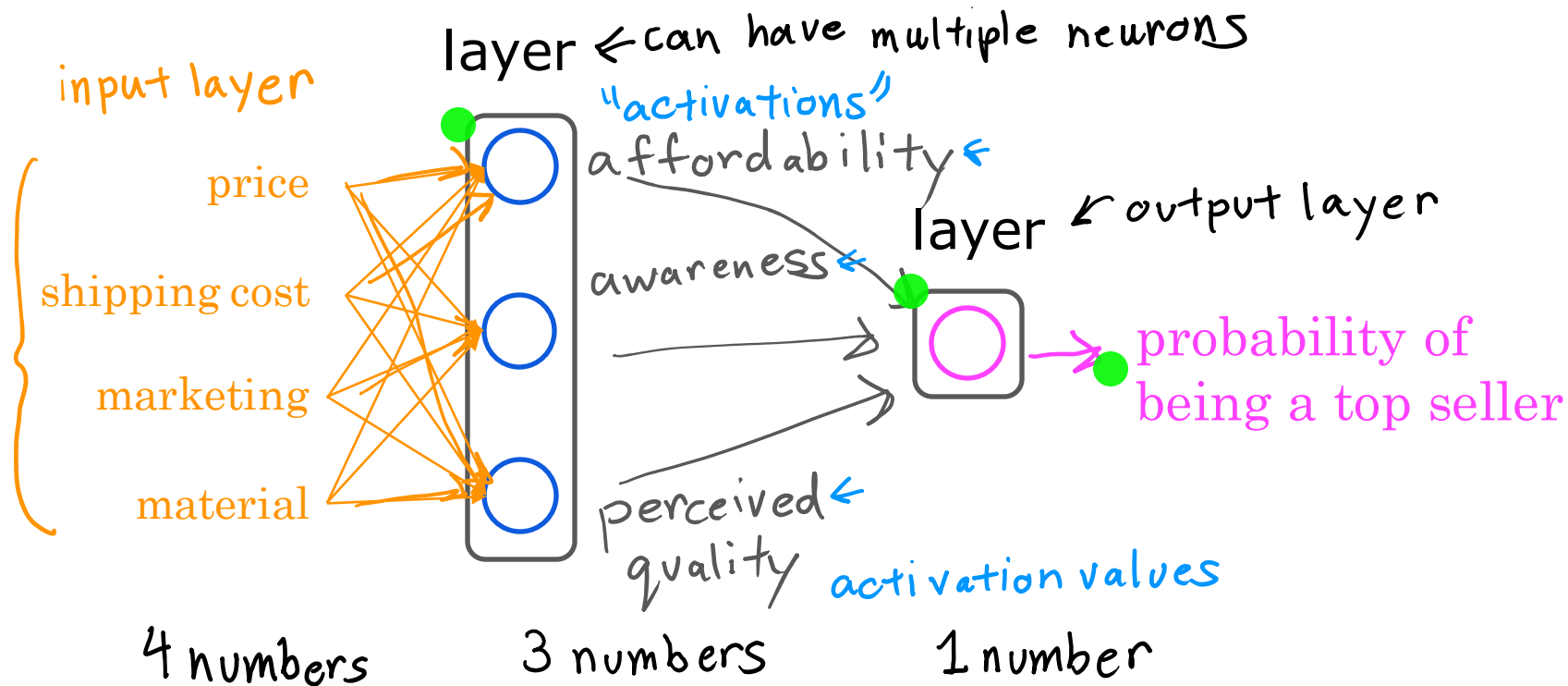
activation

input

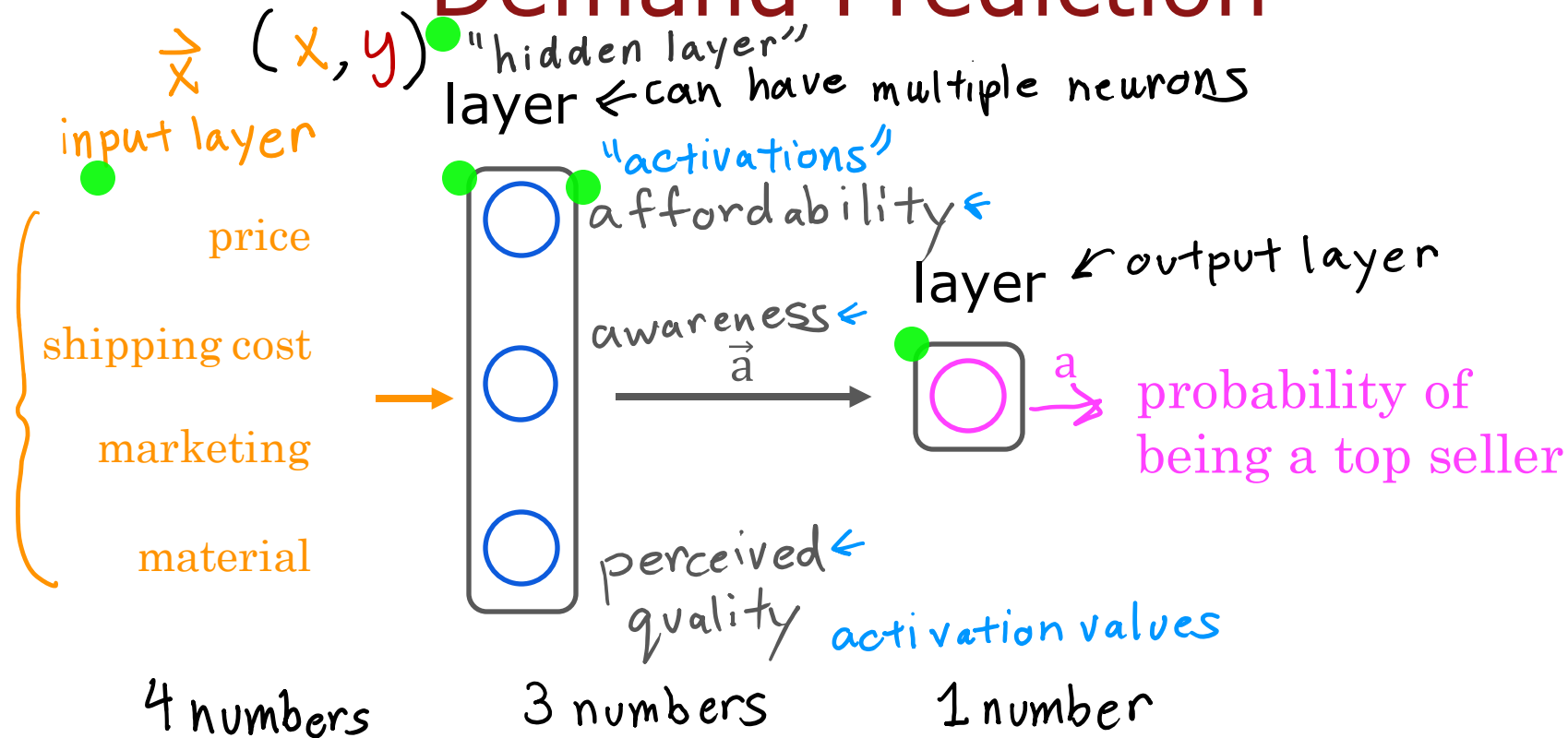
output



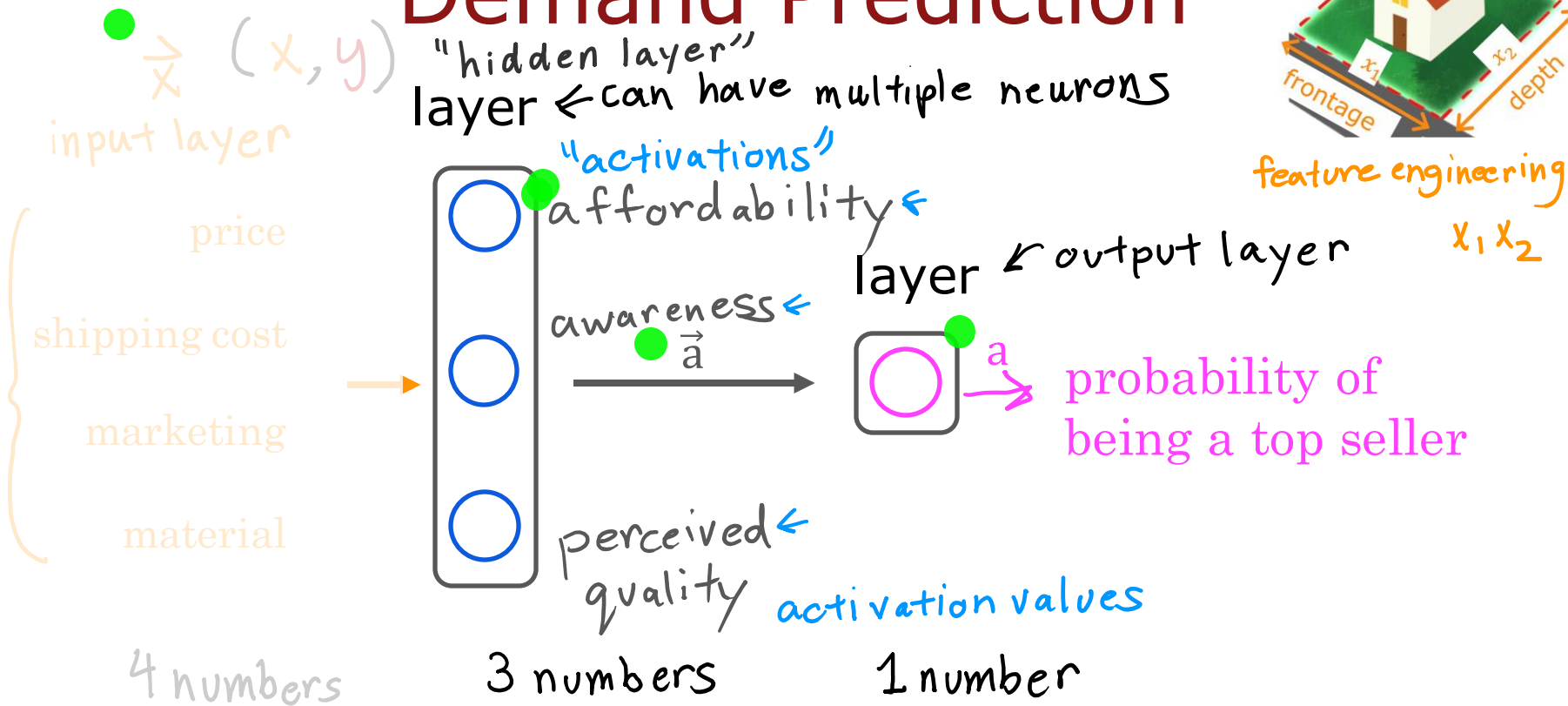
Demand Prediction



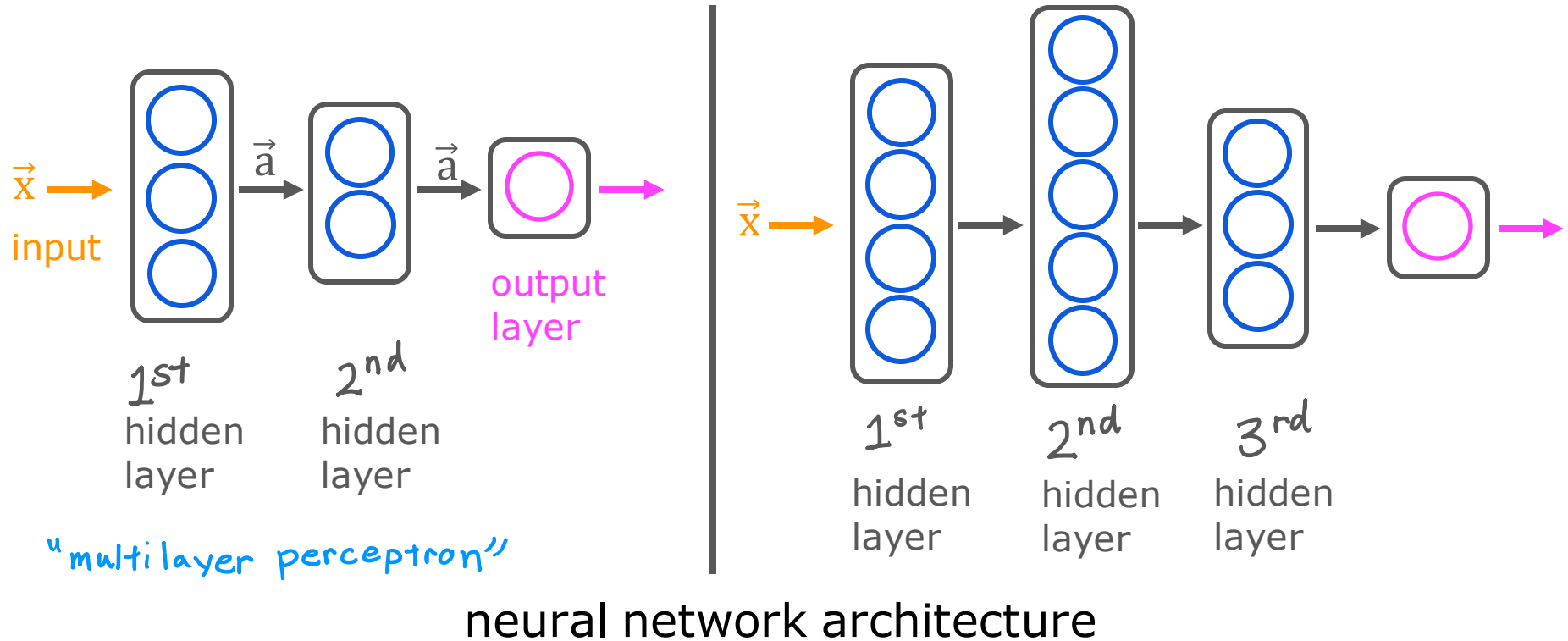
Demand Prediction



Demand Prediction

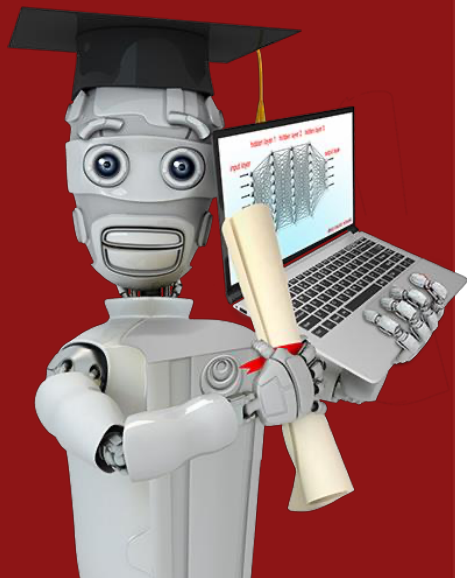


Multiple hidden layers





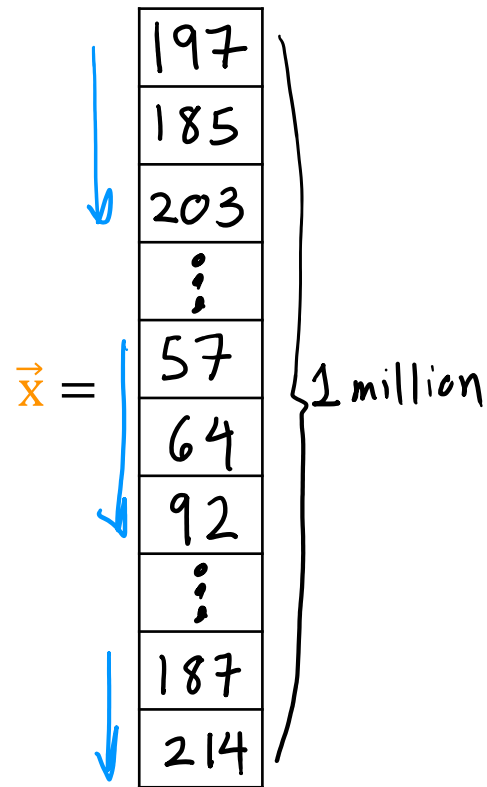
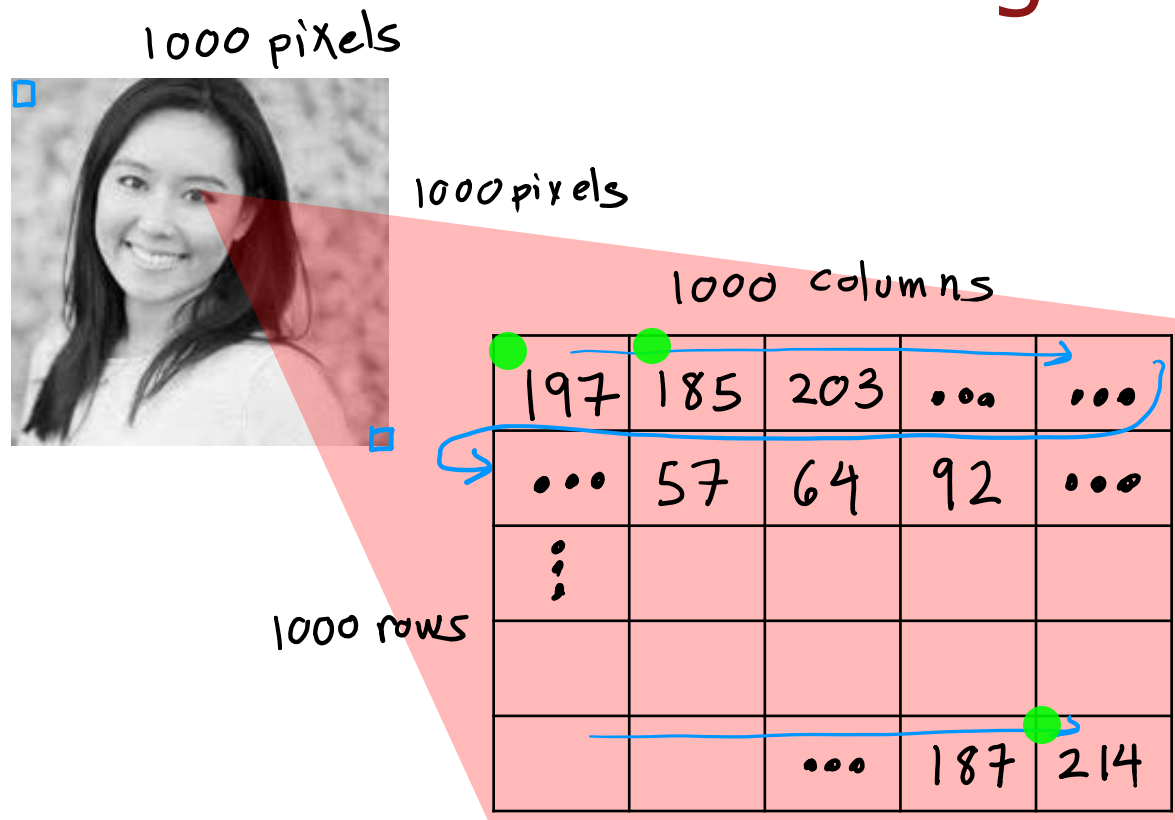
Stanford
ONLINE



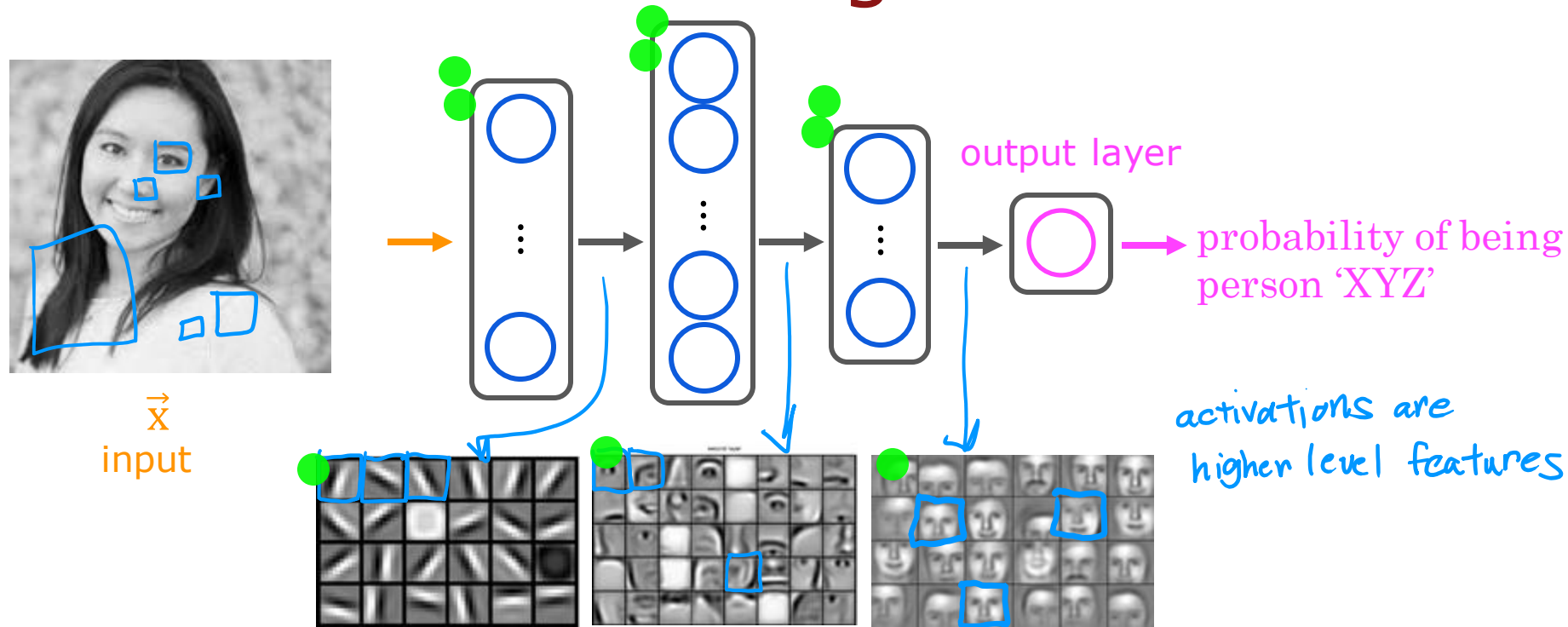
Neural Networks Intuition

**Example:
Recognizing Images**

Face recognition

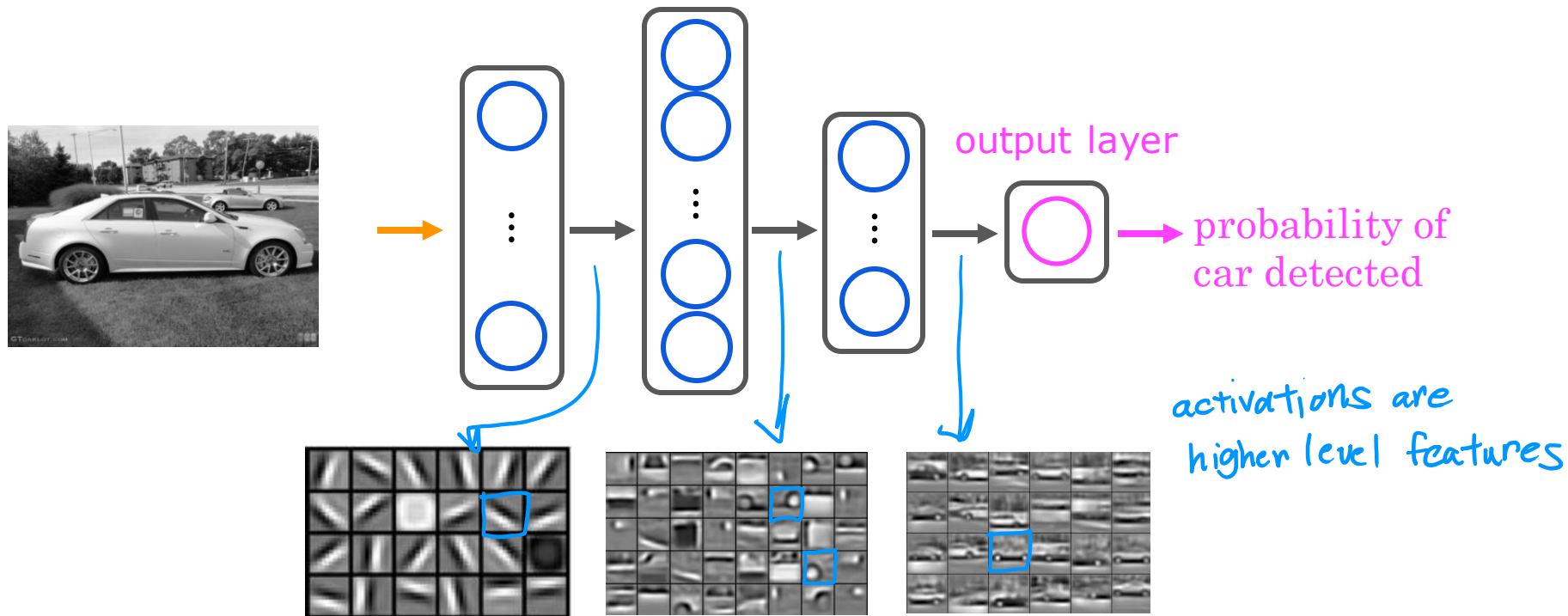


Face recognition

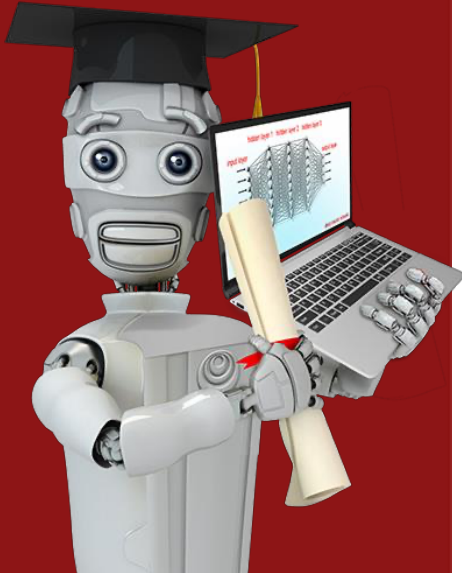


source: Convolutional Deep Belief Networks for Scalable Unsupervised Learning of Hierarchical Representations
by Honglak Lee, Roger Grosse, Ranganath Andrew Y. Ng

Car classification



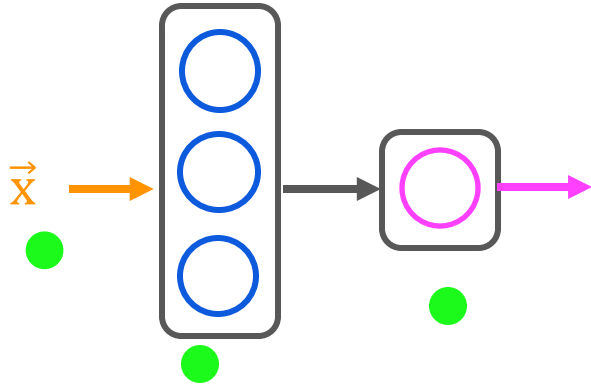
source: Convolutional Deep Belief Networks for Scalable Unsupervised Learning of Hierarchical Representations
by Honglak Lee, Roger Grosse, Ranganath Andrew Y. Ng



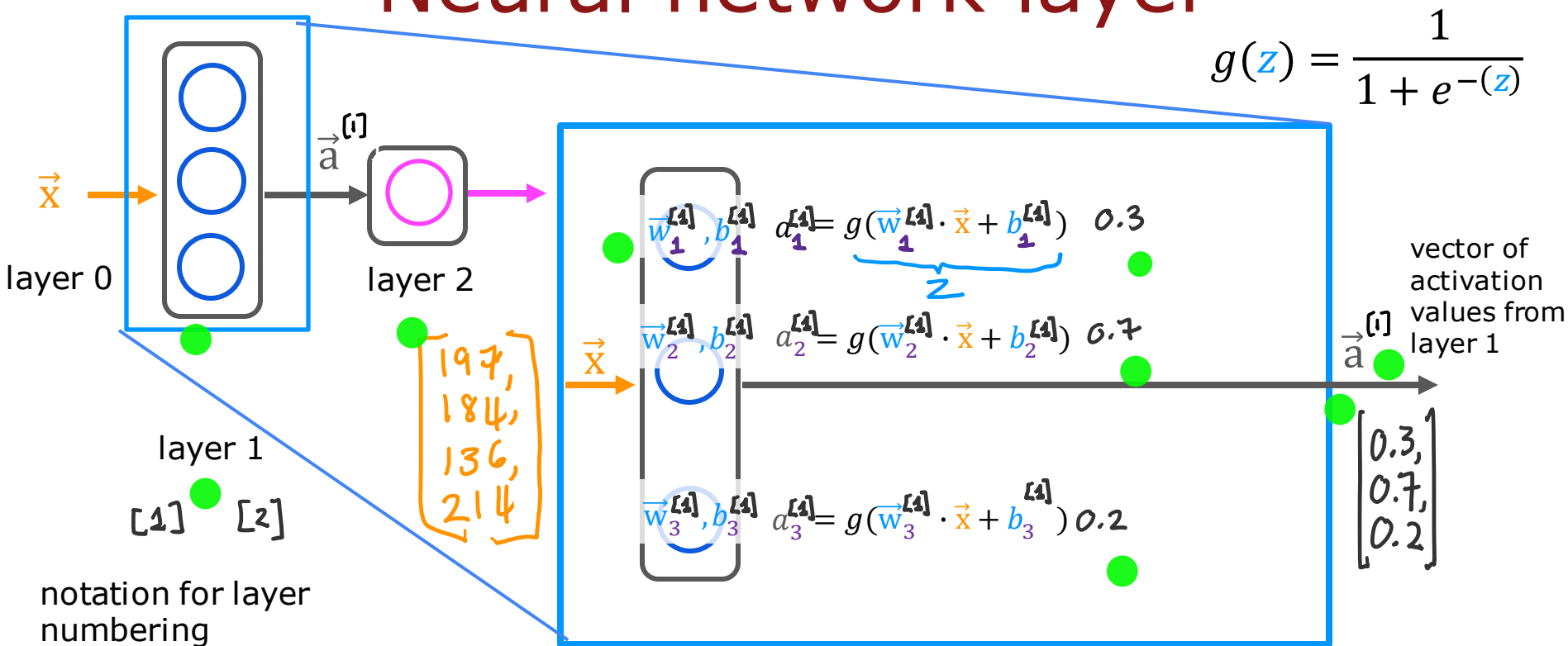
Neural network model

Neural network layer

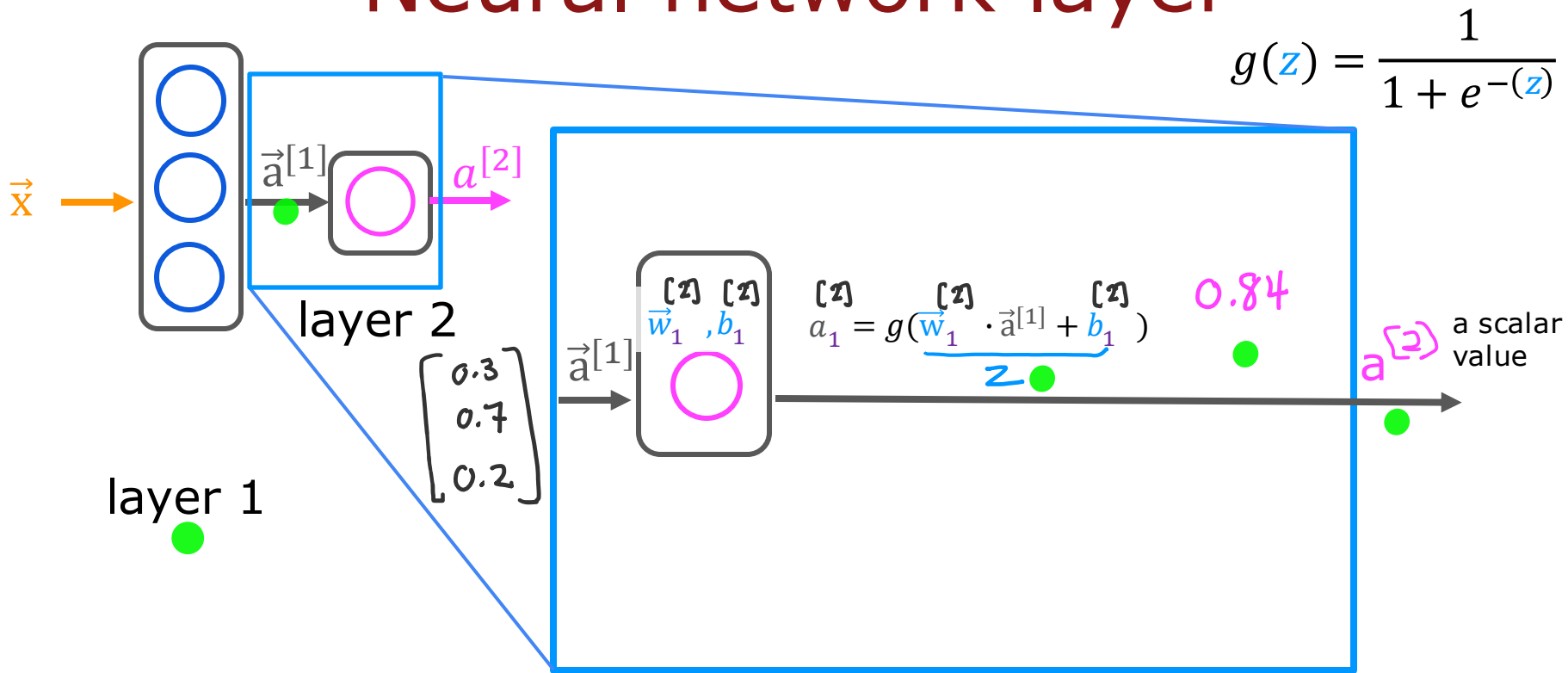
Neural network layer



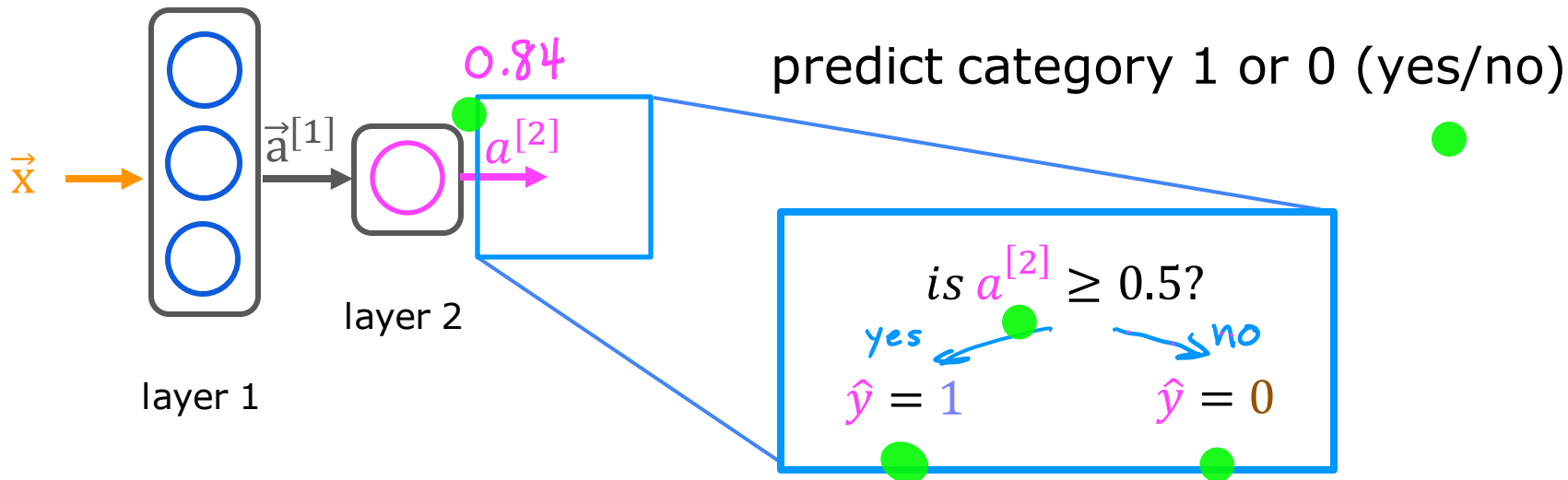
Neural network layer



Neural network layer



Neural network layer

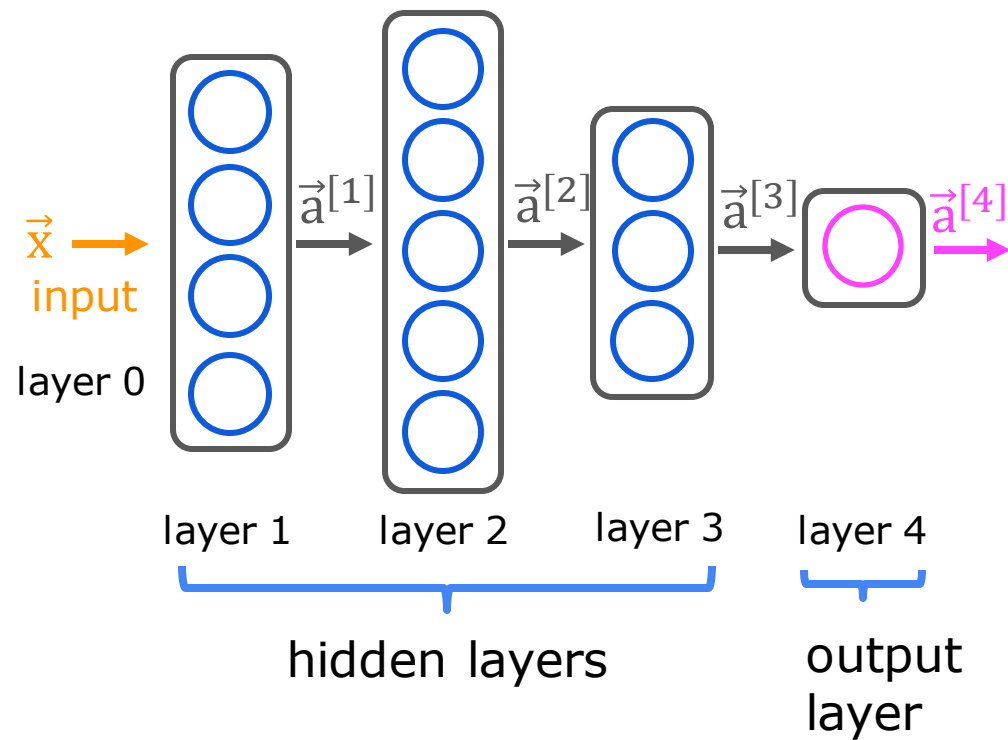




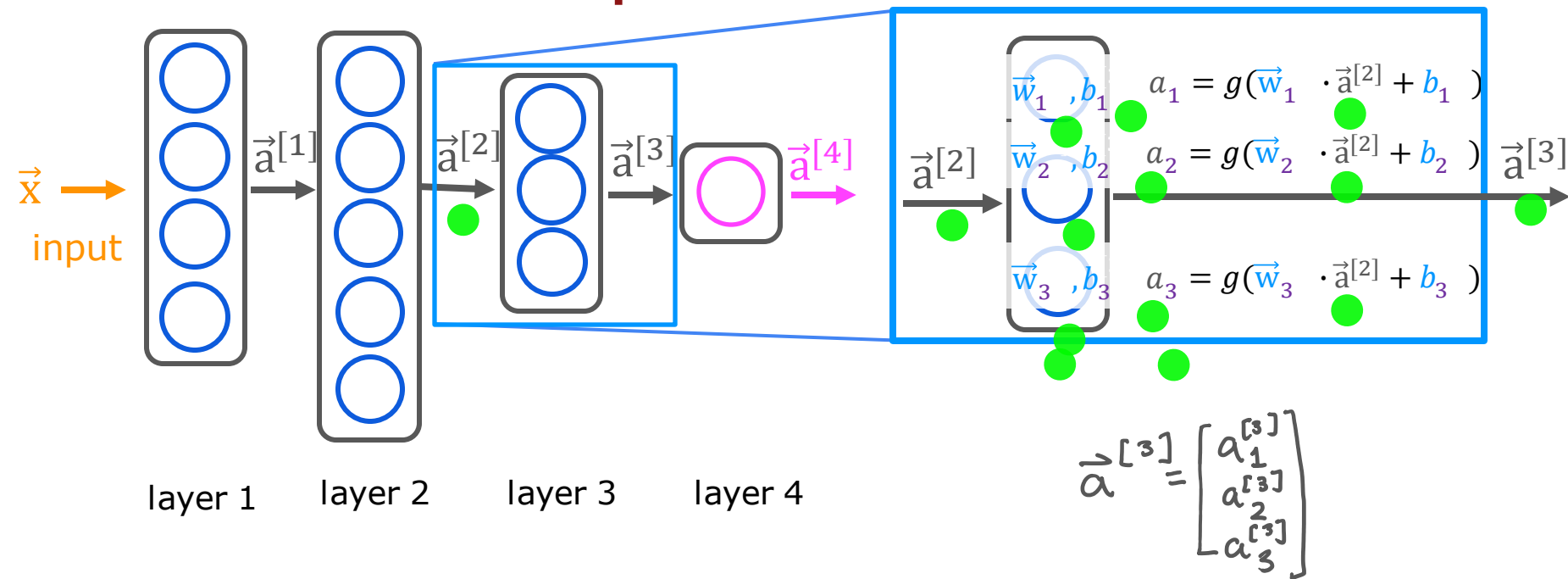
Neural Network Model

More complex neural networks

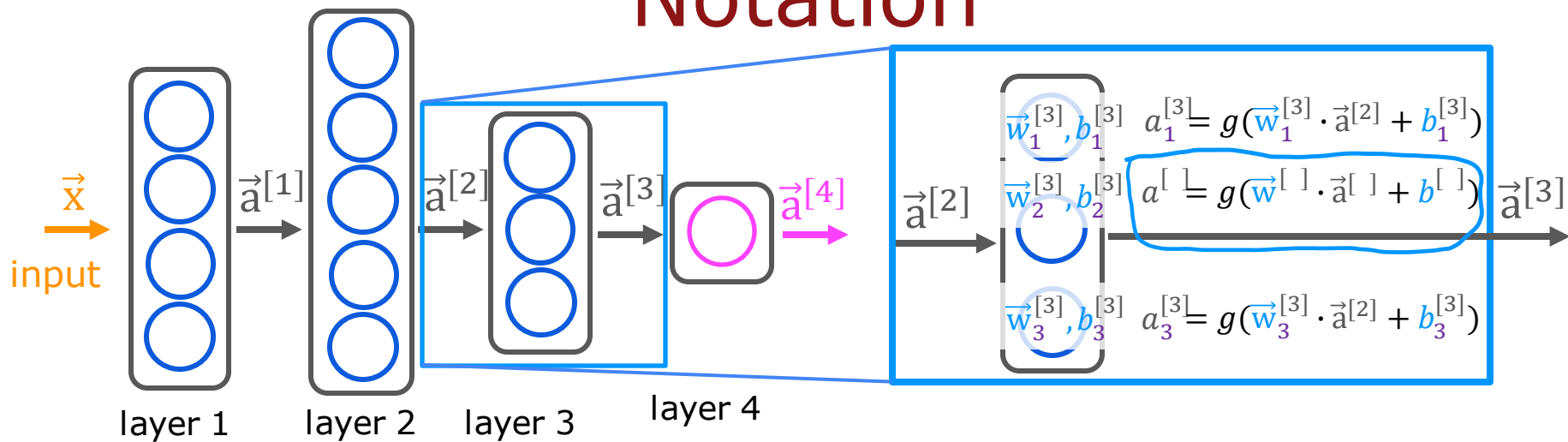
More complex neural network



More complex neural network



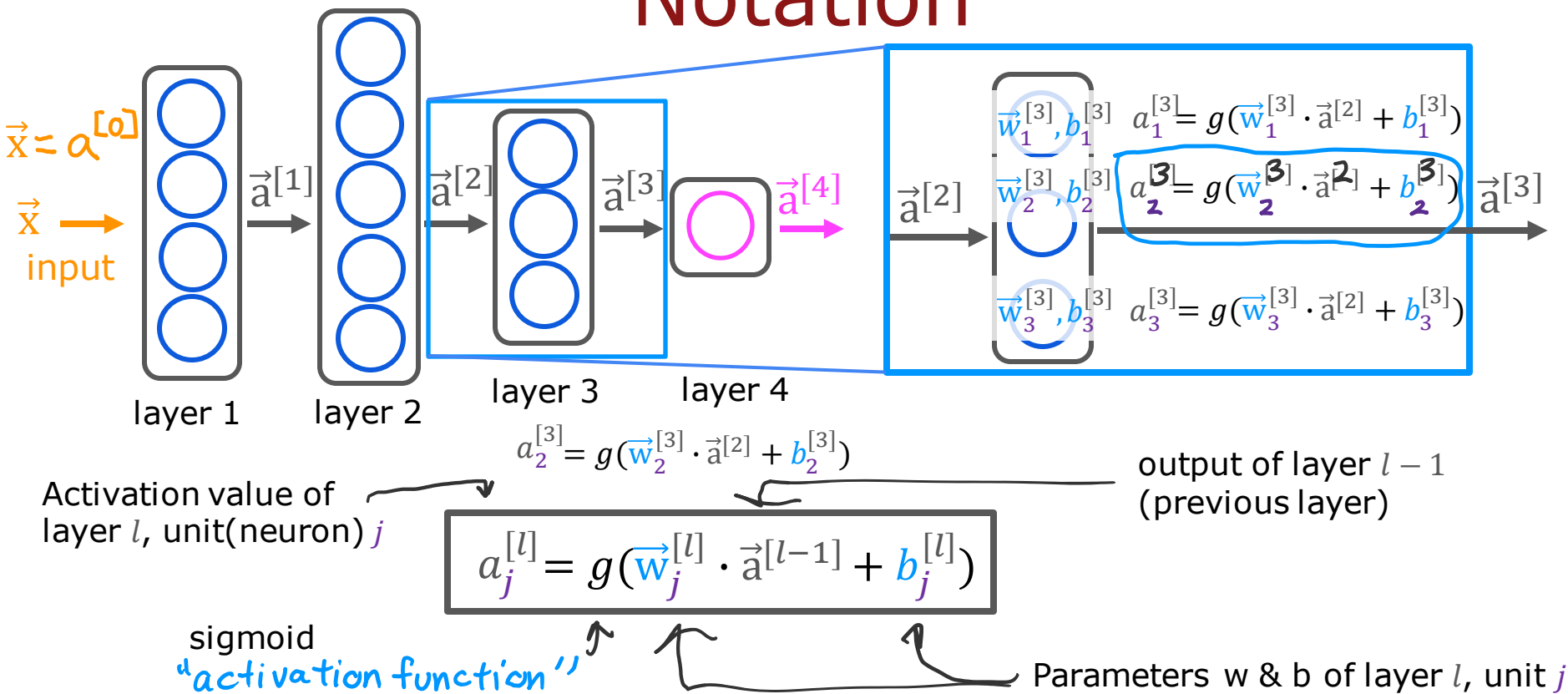
Notation

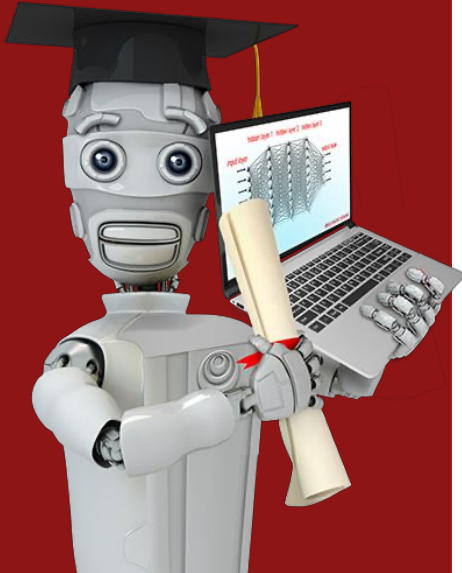


Question:

Can you fill in the superscripts and subscripts for the second neuron?

Notation

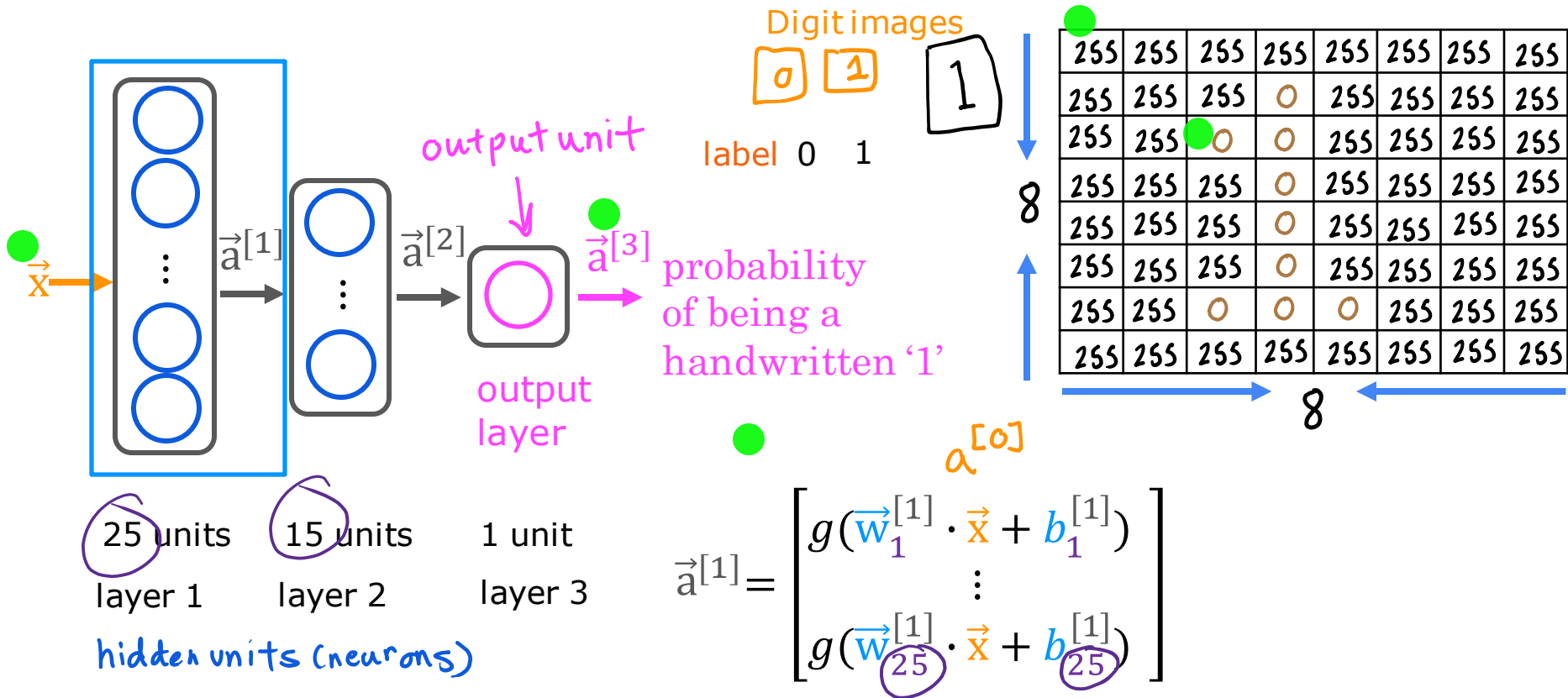




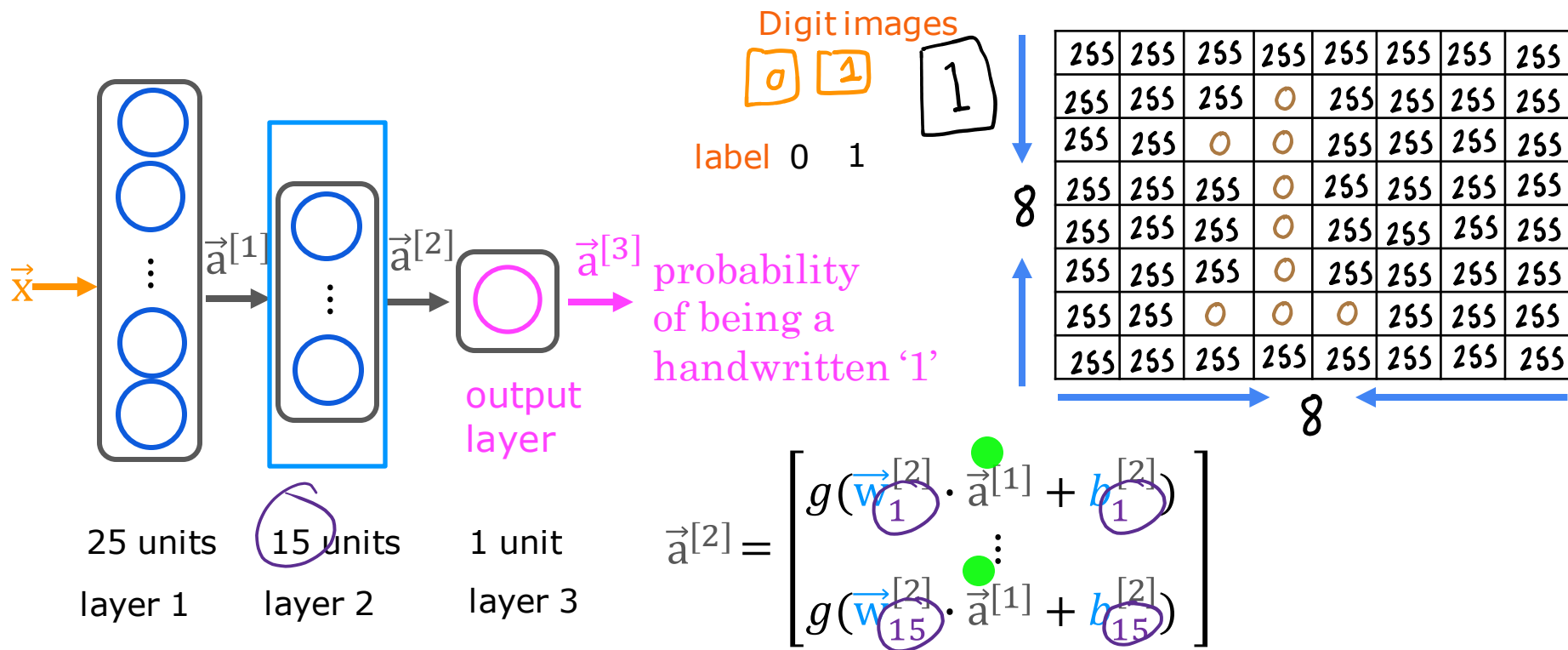
Neural Network Model

**Inference: making predictions
(forward propagation)**

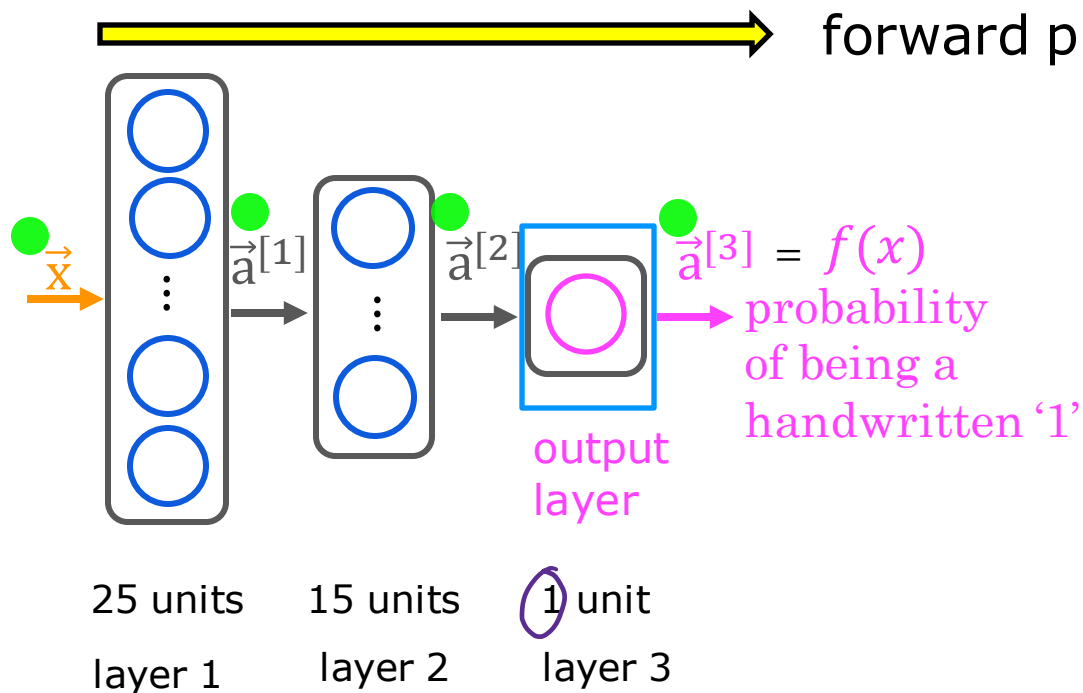
Handwritten digit recognition



Handwritten digit recognition



Handwritten digit recognition



$$\vec{a}^{[3]} = \left[g \left(\vec{w}_1^{[3]} \cdot \vec{a}^{[2]} + b_1^{[3]} \right) \right]$$

is $a_1^{[3]} \geq 0.5$?

yes

$\hat{y} = 1$

image is digit 1

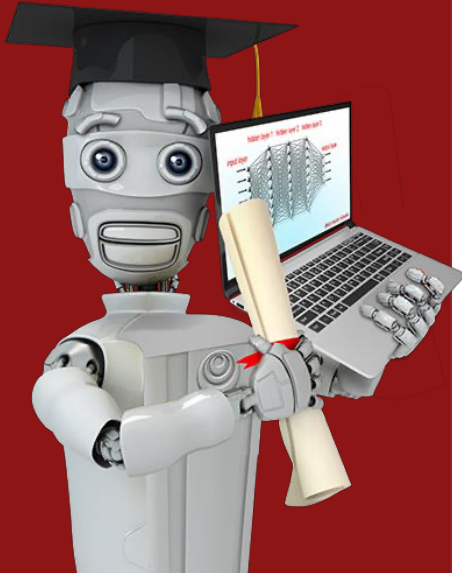
no

$\hat{y} = 0$

image isn't digit 1



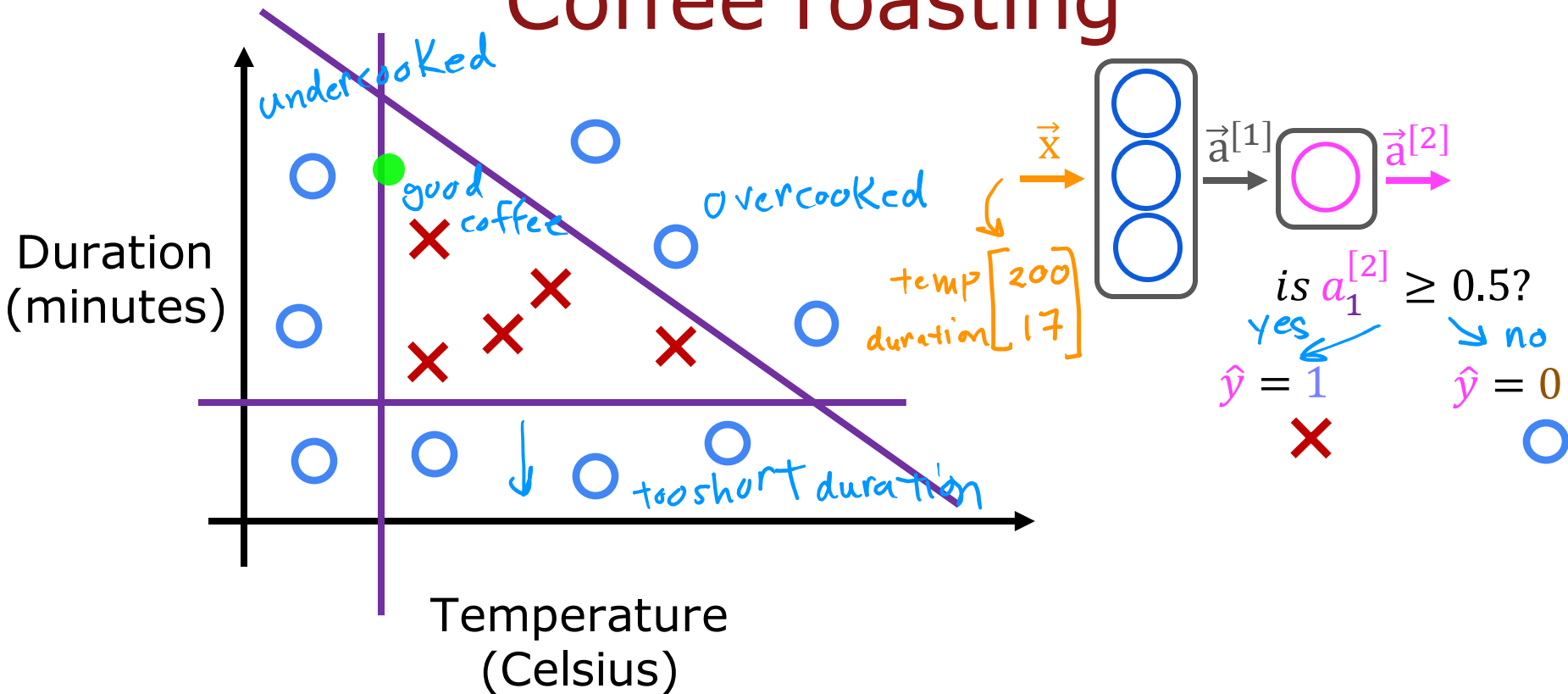
Stanford
ONLINE

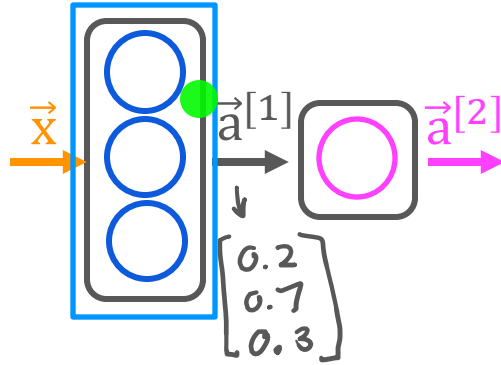


TensorFlow implementation

Inference in Code

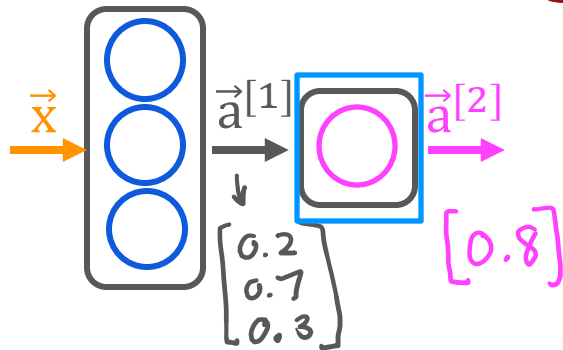
Coffee roasting





```
x = np.array([[200.0, 17.0]])  
layer_1 = Dense(units=3, activation='sigmoid')  
a1 = layer_1(x)
```

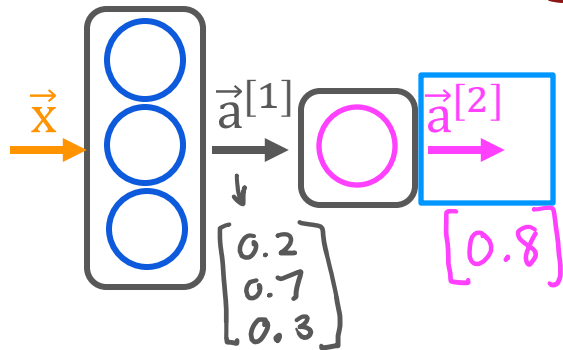
Build the model using TensorFlow



```
x = np.array([[200.0, 17.0]])  
layer_1 = Dense(units=3, activation='sigmoid')  
a1 = layer_1(x)
```

```
layer_2 = Dense(units=1, activation='sigmoid')  
a2 = layer_2(a1)
```

Build the model using TensorFlow



is $a_1^{[2]} \geq 0.5$? ●

yes $\hat{y} = 1$ ✗

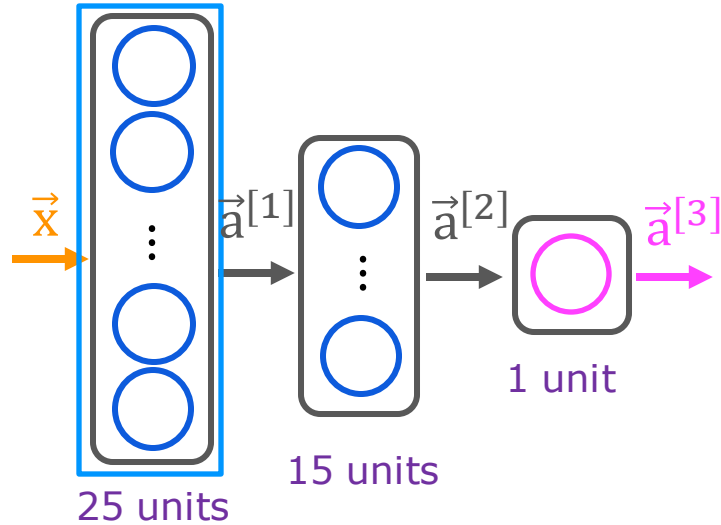
no $\hat{y} = 0$ ○

```
x = np.array([[200.0, 17.0]])  
layer_1 = Dense(units=3, activation='sigmoid')  
a1 = layer_1(x)
```

```
layer_2 = Dense(units=1, activation='sigmoid')  
a2 = layer_2(a1)
```

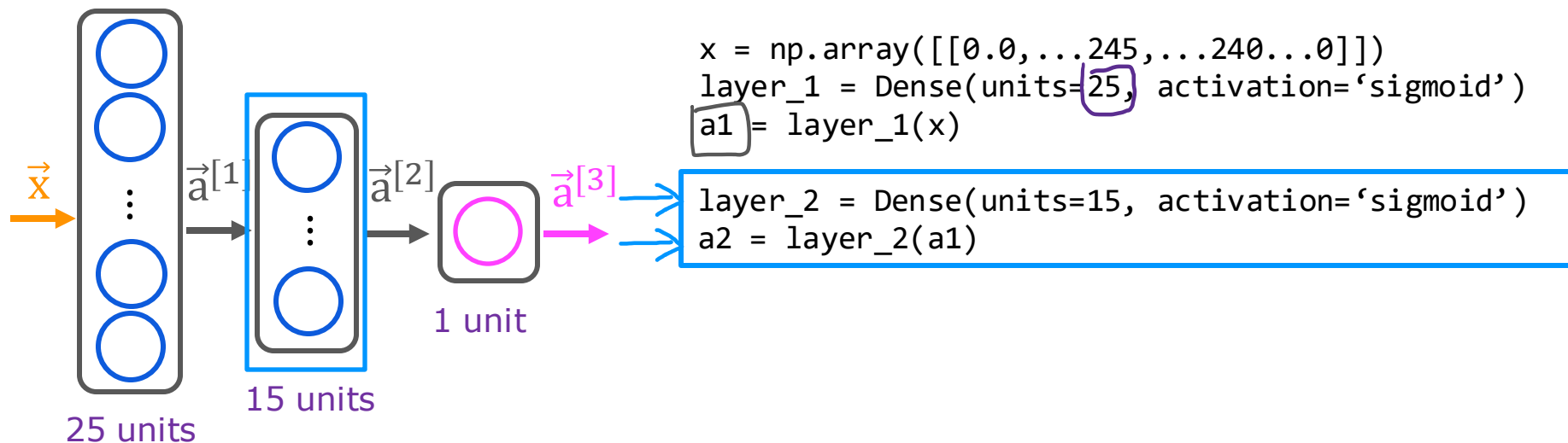
```
if a2 >= 0.5:  
    yhat = 1  
else:  
    yhat = 0
```

Model for digit classification

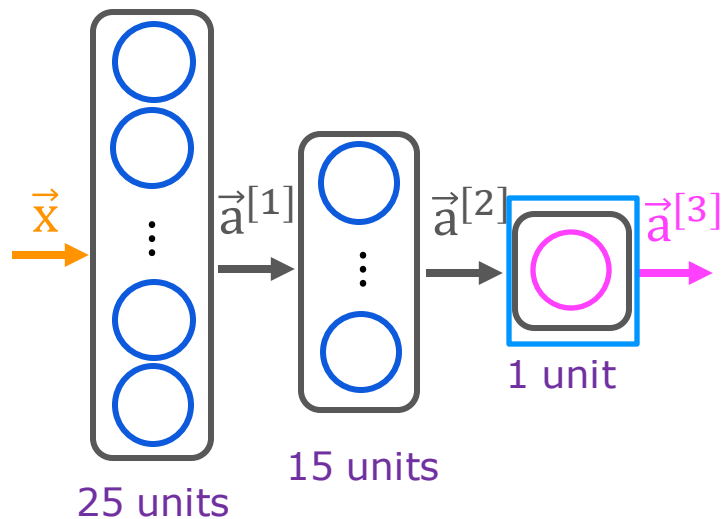


```
x = np.array([[0.0, ..., 245, ..., 240, ..., 0]])  
layer_1 = Dense(units=25, activation='sigmoid')  
a1 = layer_1(x)
```

Model for digit classification



Model for digit classification

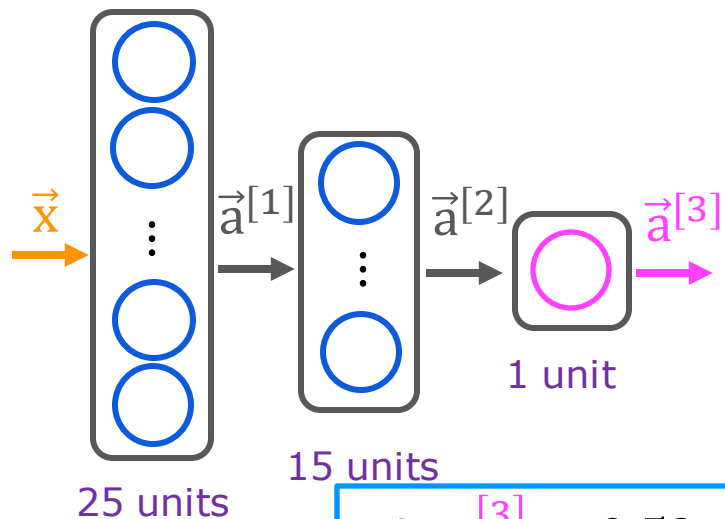


```
x = np.array([[0.0, ..., 245, ..., 240, ..., 0]])  
layer_1 = Dense(units=25, activation='sigmoid')  
a1 = layer_1(x)
```

```
layer_2 = Dense(units=15, activation='sigmoid')  
a2 = layer_2(a1)
```

```
layer_3 = Dense(units=1, activation='sigmoid')  
a3 = layer_3(a2)
```

Model for digit classification



```
x = np.array([[0.0, ..., 245, ..., 240, ..., 0]])  
layer_1 = Dense(units=25, activation='sigmoid')  
a1 = layer_1(x)
```

```
layer_2 = Dense(units=15, activation='sigmoid')  
a2 = layer_2(a1)
```

```
layer_3 = Dense(units=1, activation='sigmoid')  
a3 = layer_3(a2)
```

is $a_1^{[3]} \geq 0.5$?

$\hat{y} = 1$

✗

$\hat{y} = 0$

○

```
if a3 >= 0.5:  
    yhat = 1  
else:  
    yhat = 0
```



TensorFlow implementation

Data in TensorFlow

Feature vectors

temperature (Celsius)	duration (minutes)	Good coffee? (1/0)
200.0	17.0	1
425.0	18.5	0
...	...	...

```
x = np.array([[200.0, 17.0]])
```

←

[[200.0, 17.0]]

why?

Note about numpy arrays

2 rows
3 columns
 $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$
2 x 3 matrix

4 rows
2 columns
 $\begin{bmatrix} 0.1 & 0.2 \\ -3 & -4 \\ -0.5 & -0.6 \\ 7 & 8 \end{bmatrix}$
4 x 2 matrix

$x = \text{np.array}(\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix})$
 $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$

$x = \text{np.array}(\begin{bmatrix} 0.1 & 0.2 \\ -3.0 & -4.0 \\ -0.5 & -0.6 \\ 7.0 & 8.0 \end{bmatrix})$
 $\begin{bmatrix} 0.1 & 0.2 \\ -3.0 & -4.0 \\ -0.5 & -0.6 \\ 7.0 & 8.0 \end{bmatrix}$

2D array

2 x 3

4 x 2

1 x 2

2 x 1

Note about numpy arrays

`x = np.array([[200, 17]])` → $\begin{bmatrix} 200 & 17 \end{bmatrix}$ 1 x 2

`x = np.array([200, 17])` → $\begin{bmatrix} 200 \\ 17 \end{bmatrix}$ 2 x 1

→ `x = np.array([200, 17])`

1D
"Vector"

Feature vectors

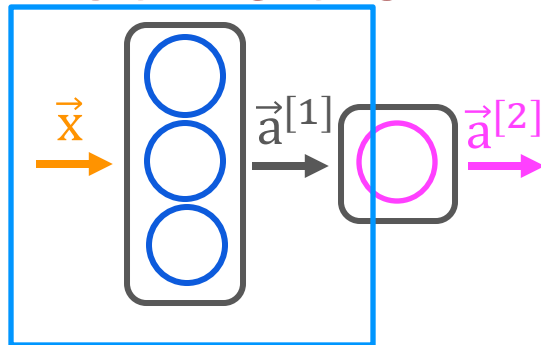
temperature (Celsius)	duration (minutes)	Good coffee? (1/0)
200.0	17.0	1
425.0	18.5	0
...	...	...

`x = np.array([[200.0, 17.0]])` ←

`[[200.0, 17.0]]`

→ $\begin{matrix} \downarrow & \downarrow \\ [200.0 & 17.0] \end{matrix}$ 1 x 2

Activation vector



```
x = np.array([[200.0, 17.0]])  
layer_1 = Dense(units=3, activation='sigmoid')  
a1 = layer_1(x)
```

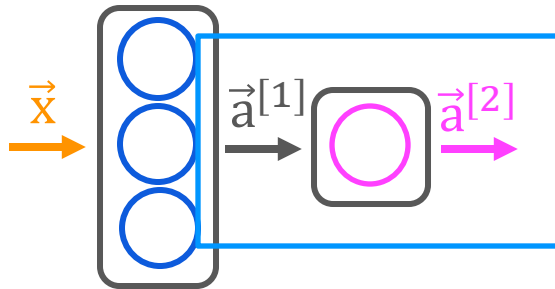
→ $[0.2, 0.7, 0.3]$ 1 x 3 matrix

→ `tf.Tensor([[0.2 0.7 0.3]], shape=(1, 3), dtype=float32)`

→ `a1.numpy()`

`array([[1.4661001, 1.125196 , 3.2159438]], dtype=float32)`

Activation vector



```
→ layer_2 = Dense(units=1, activation='sigmoid')
→ a2 = layer_2(a1)
```

↖ $[[0.8]]$ ↗

```
→ tf.Tensor([[0.8]], shape=(1, 1), dtype=float32)
```

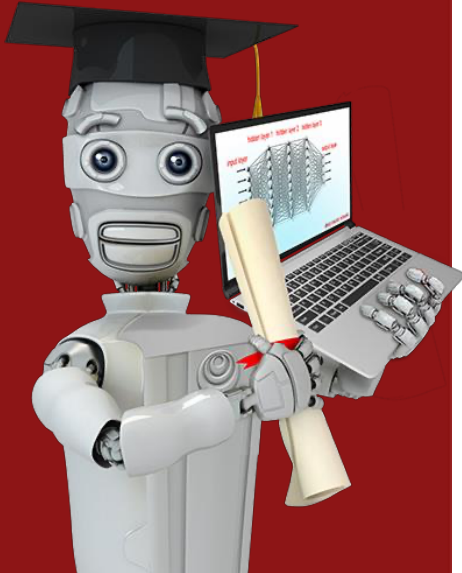
```
→ a2.numpy()
```

```
→ array([[0.8]], dtype=float32)
```

1 x 1



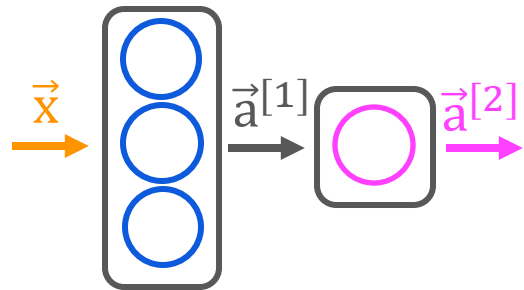
Stanford
ONLINE



TensorFlow implementation

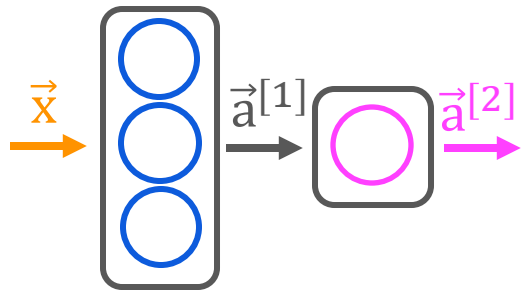
Building a neural network

What you saw earlier



```
→ x = np.array([[200.0, 17.0]])  
→ layer_1 = Dense(units=3, activation="sigmoid")  
→ a1 = layer_1(x)  
  
→ layer_2 = Dense(units=1, activation="sigmoid")  
→ a2 = layer_2(a1)
```

Building a neural network architecture



```
→ layer_1 = Dense(units=3, activation="sigmoid")  
→ layer_2 = Dense(units=1, activation="sigmoid")  
→ model = Sequential([layer_1, layer_2])
```

layer1
layer2

```
x = np.array([[200.0, 17.0],  
              [120.0, 5.0],  
              [425.0, 20.0],  
              [212.0, 18.0]])
```

4 x 2

targets

```
y = np.array([1,0,0,1])
```

```
model.compile(...)
```

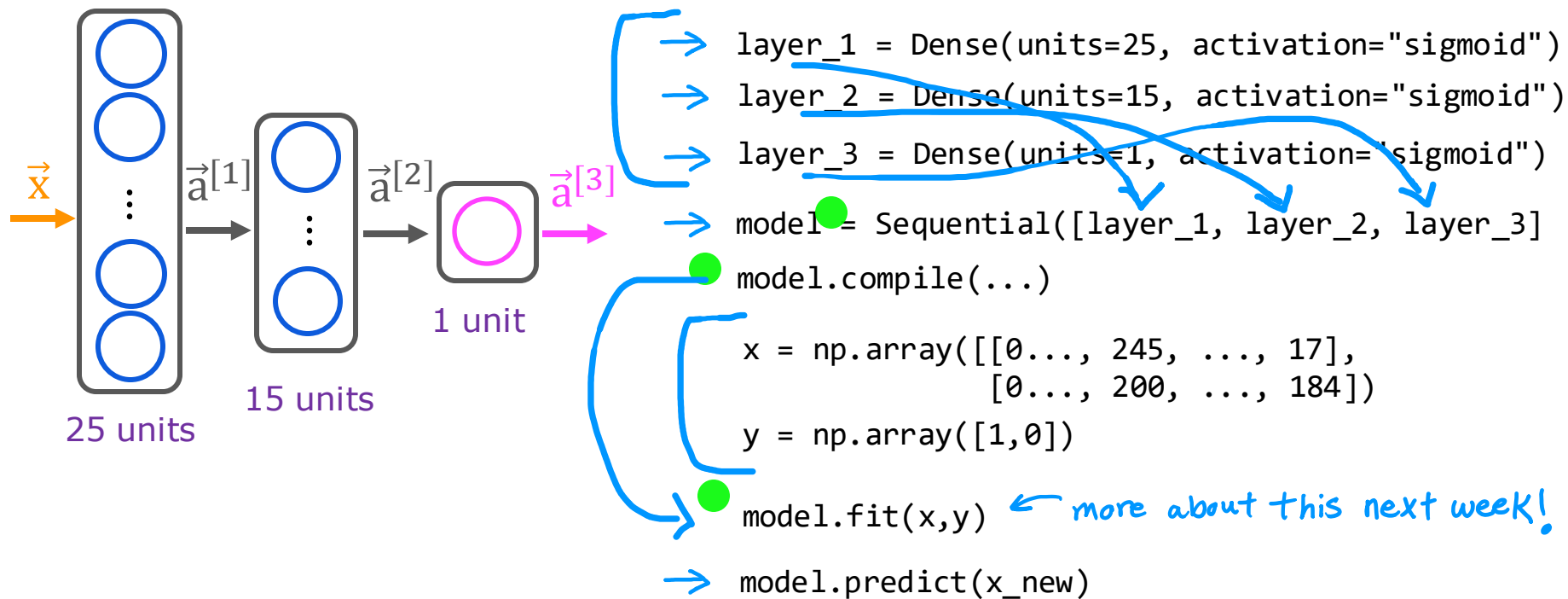
```
model.fit(x,y)
```

```
→ model.predict(x_new)
```

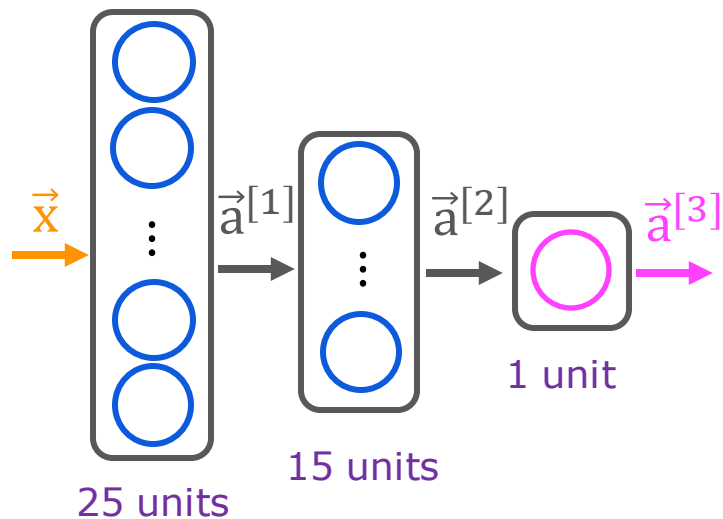
← more about this next week!

			y
●	200	17	1
	120	5	0
	425	20	0
	212	18	1

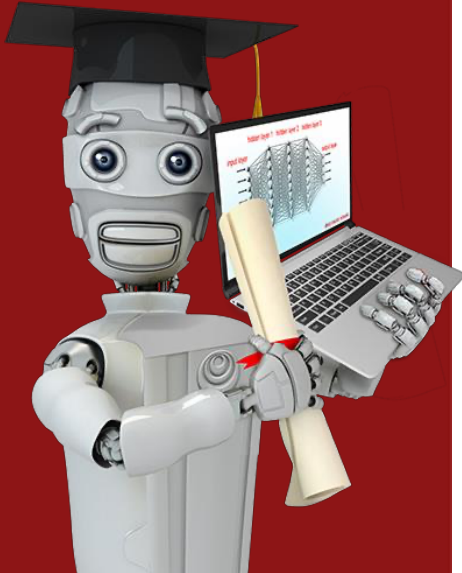
Digit classification model



Digit classification model



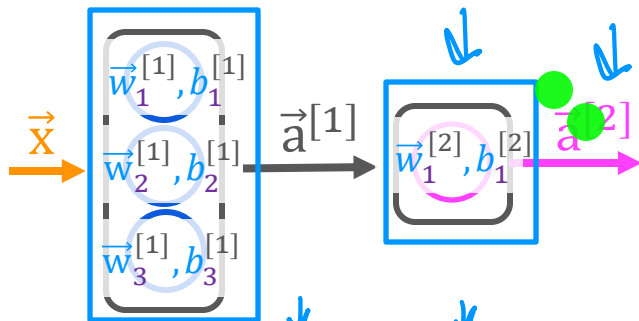
```
model = Sequential([  
    → Dense(units=25, activation="sigmoid"),  
    → Dense(units=15, activation="sigmoid"),  
    → Dense(units=1, activation="sigmoid")])  
  
model.compile(...)  
  
x = np.array([[0..., 245, ..., 17],  
              [0..., 200, ..., 184]])  
y = np.array([1,0])  
  
model.fit(x,y)  
  
model.predict(x_new)
```



Neural network implementation in Python

Forward prop in a single layer

forward prop (coffee roasting model)



$x = \text{np.array}([200, 17])$

$$a_1^{[1]} = g(\vec{w}_1^{[1]} \cdot \vec{x} + b_1^{[1]})$$

$$a_2^{[1]} = g(\vec{w}_2^{[1]} \cdot \vec{x} + b_2^{[1]})$$

$$a_3^{[1]} = g(\vec{w}_3^{[1]} \cdot \vec{x} + b_3^{[1]})$$

$w1_1 = \text{np.array}([1, 2])$

$w1_2 = \text{np.array}([-3, 4])$

$w1_3 = \text{np.array}([5, -6])$

$b1_1 = \text{np.array}([-1])$

$b1_2 = \text{np.array}([1])$

$b1_3 = \text{np.array}([2])$

$z1_1 = \text{np.dot}(w1_1, x) + b1_1$

$z1_2 = \text{np.dot}(w1_2, x) + b1_2$

$z1_3 = \text{np.dot}(w1_3, x) + b1_3$

$a1_1 = \text{sigmoid}(z1_1)$

$a1_2 = \text{sigmoid}(z1_2)$

$a1_3 = \text{sigmoid}(z1_3)$

$a1 = \text{np.array}([a1_1, a1_2, a1_3])$

$$a_1^{[2]} = g(\vec{w}_1^{[2]} \cdot \vec{a}^{[1]} + b_1^{[2]})$$

$w2_1 = \text{np.array}([-7, 8])$

$b2_1 = \text{np.array}([3])$

$z2_1 = \text{np.dot}(w2_1, a1) + b2_1$

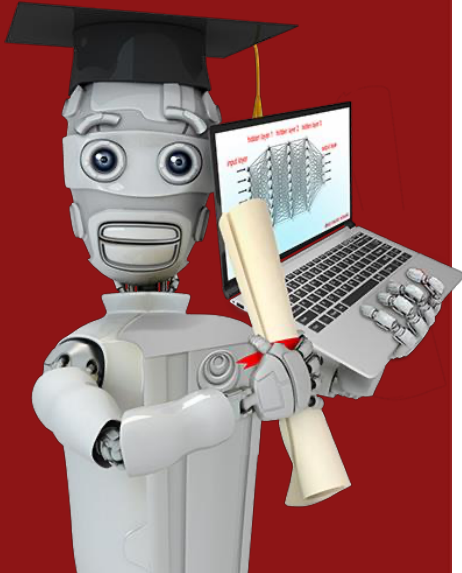
$a2_1 = \text{sigmoid}(z2_1)$

$w_1^{[2]}$ $w2_1$

1D arrays



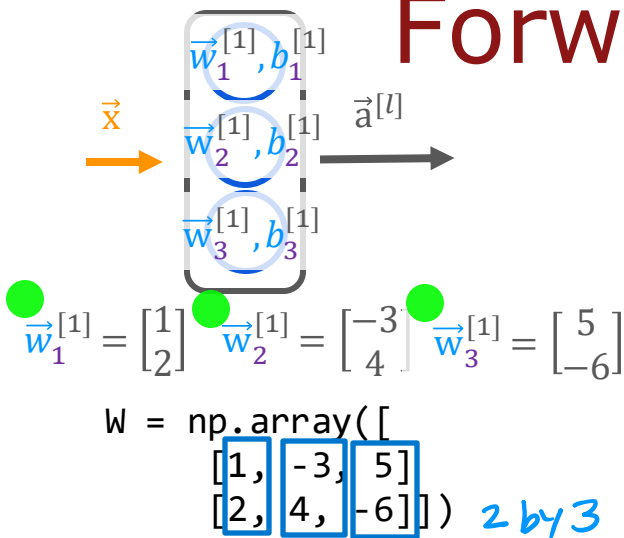
Stanford
ONLINE



Neural network implementation in Python

General implementation of
forward propagation

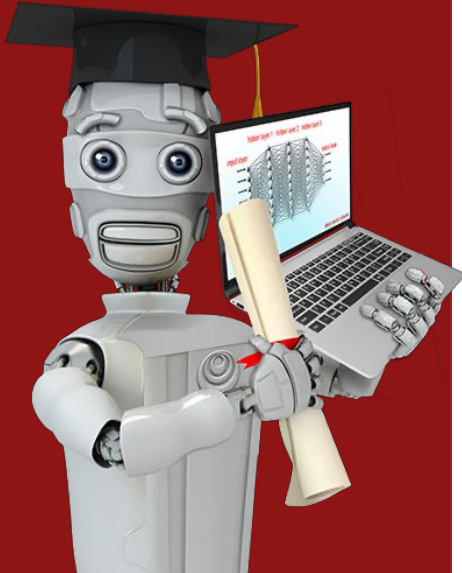
Forward prop in NumPy



```
def dense(a_in, W, b, g):
    units = W.shape[1]
    a_out = np.zeros(units)
    for j in range(units):
        w = W[:, j]
        z = np.dot(w, a_in) + b[j]
        a_out[j] = g(z)
    return a_out
```

```
def sequential(x):
    a1 = dense(x, W1, b1, g)
    a2 = dense(a1, W2, b2, g)
    a3 = dense(a2, W3, b3, g)
    a4 = dense(a3, W4, b4, g)
    f_x = a4
    return f_x
```

capital W refers to a matrix



Speculations on artificial general intelligence (AGI)

Is there a path to AGI?

AI

```
graph TD; AI[AI] --- ANI[ANI]; AI --- AGI[AGI];
```

ANI

(artificial narrow intelligence)

E.g., smart speaker,
self-driving car, web search,
AI in farming and factories

AGI

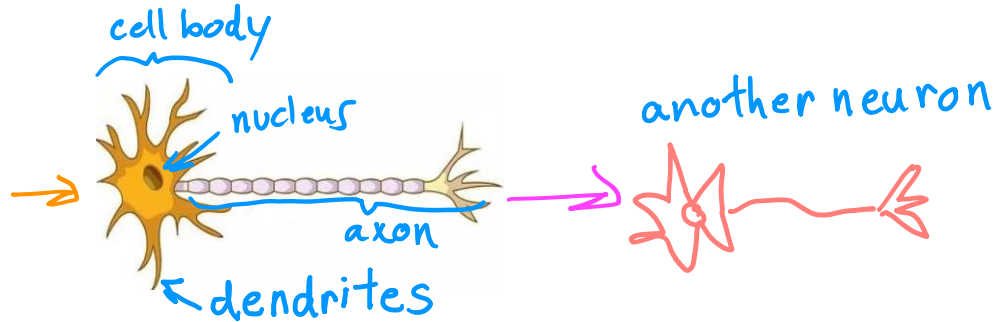
(artificial general intelligence)

Do anything a human can do

Biological neuron

inputs

outputs



Simplified mathematical model of a neuron

inputs

outputs

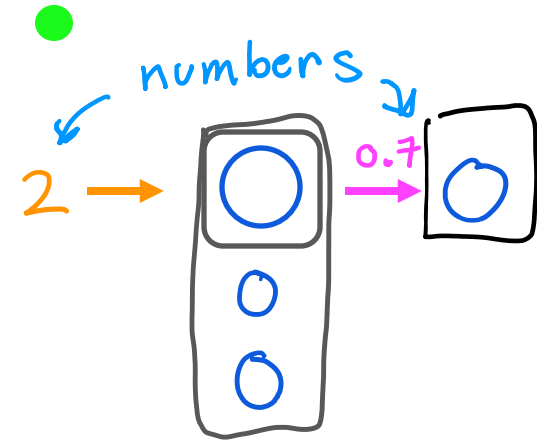
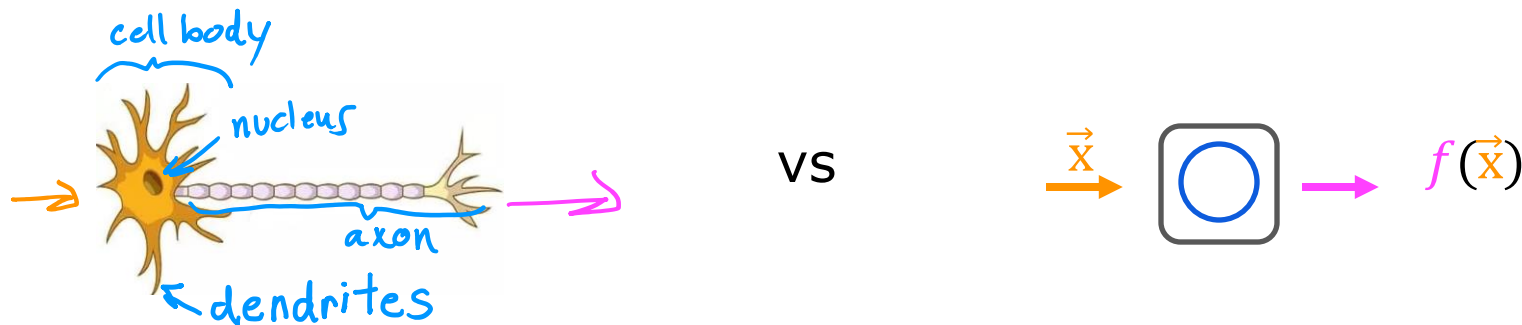


image source: <https://biologydictionary.net/sensory-neuron/>

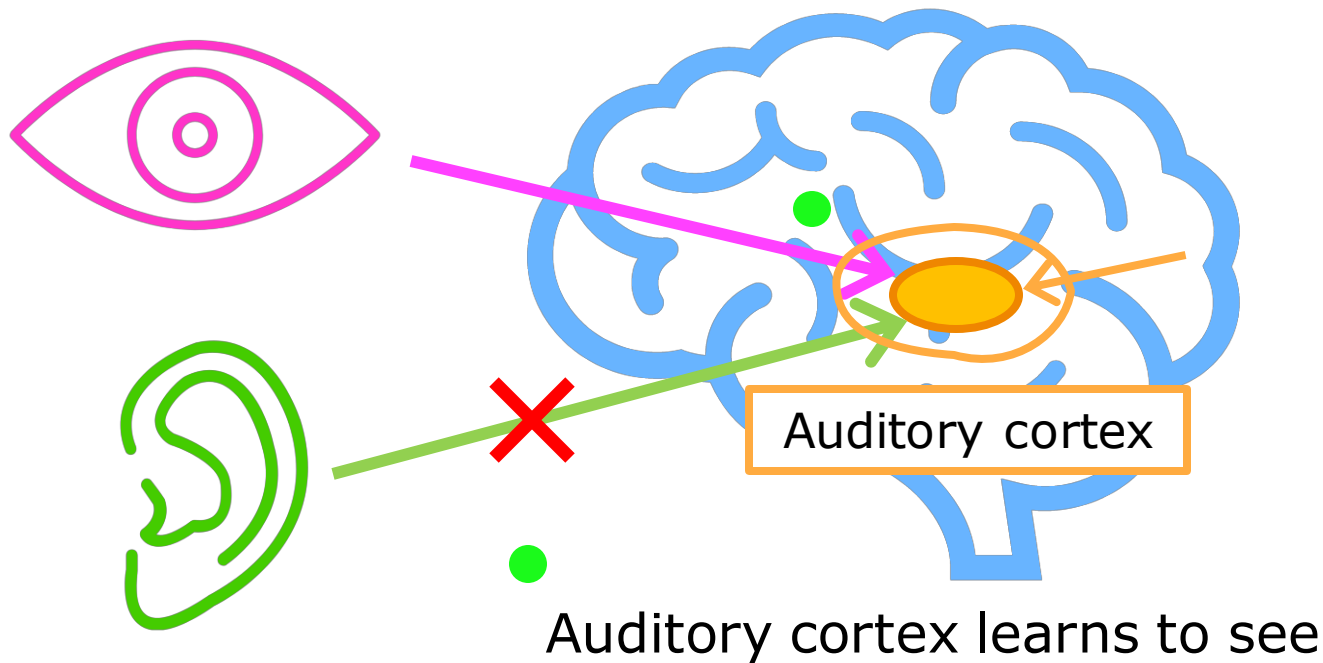
Neural network and the brain

Can we mimic the human brain?



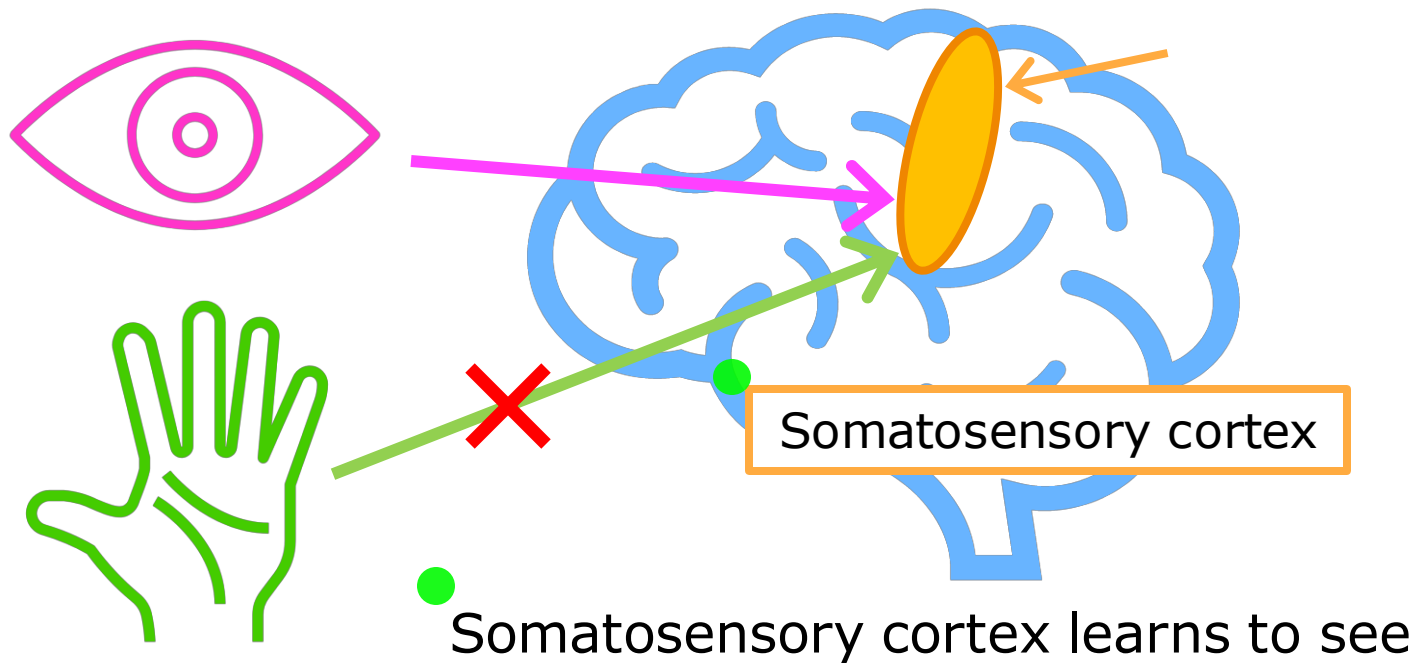
We have (almost) no idea how the brain works

The "one learning algorithm" hypothesis



[Roe et al., 1992]

The "one learning algorithm" hypothesis



[Metin & Frost, 1989]

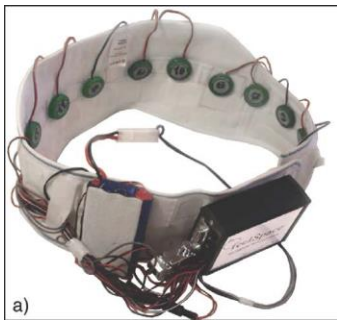
Sensor representations in the brain



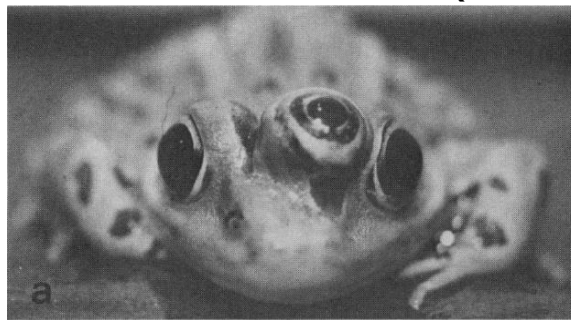
Seeing with your tongue



Human echolocation (sonar)



Haptic belt: Direction sense



Implanting a 3rd eye

[BrainPort; Welsh & Blasch, 1997; Nagel et al., 2005; Constantine-Paton & Law, 2009]



Stanford
ONLINE



Vectorization (optional)

How neural networks are
implemented efficiently

For loops vs. vectorization

```
x = np.array([200, 17])
W = np.array([[1, -3, 5],
              [-2, 4, -6]])
b = np.array([-1, 1, 2])

def dense(a_in, W, b):
    a_out = np.zeros(units)
    for j in range(units):
        w = W[:,j]
        z = np.dot(w, x) + b[j]
        a[j] = g(z)
    return a
```

[1,0,1]

```
X = np.array([[200, 17]])
W = np.array([[1, -3, 5],
              [-2, 4, -6]])
B = np.array([[-1, 1, 2]])

def dense(A_in, W, B):
    Z = np.matmul(A_in, W) + B
    A_out = g(Z)
    return A_out

[[1,0,1]]
```

vectorized

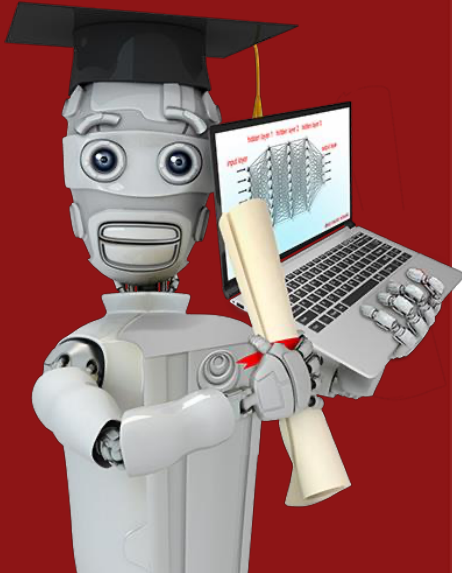
2D array

same

1x3 2D array

all 2D arrays

matrix multiplication

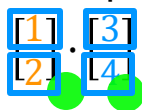


Vectorization (optional)

Matrix multiplication

Dot products

example

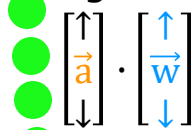

$$\begin{bmatrix} 1 \\ 2 \end{bmatrix} \cdot \begin{bmatrix} 3 \\ 4 \end{bmatrix}$$

$$z = (1 \times 3) + (2 \times 4)$$

3 + 8

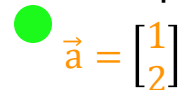
11

in general


$$\begin{bmatrix} \uparrow \\ \downarrow \end{bmatrix} \vec{a} \cdot \begin{bmatrix} \uparrow \\ \downarrow \end{bmatrix} \vec{w}$$

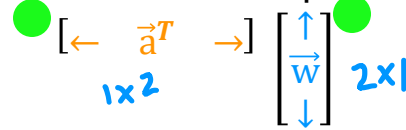
$$z = \vec{a} \cdot \vec{w}$$

transpose


$$\vec{a} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$


$$\vec{a}^T = \begin{bmatrix} 1 & 2 \end{bmatrix}$$

vector vector multiplication


$$\begin{bmatrix} \leftarrow & \vec{a}^T & \rightarrow \end{bmatrix} \begin{bmatrix} \uparrow \\ \downarrow \end{bmatrix} \vec{w}$$

1×2 2×1

$$z = \vec{a}^T \vec{w}$$

equivalent

useful for understanding
matrix multiplication

Vector matrix multiplication

$$\vec{a} = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \quad \vec{a}^T = \begin{bmatrix} 1 & 2 \end{bmatrix} \quad \mathbf{W} = \begin{bmatrix} 3 & 5 \\ 4 & 6 \end{bmatrix} \quad \mathbf{Z} = \vec{a}^T \mathbf{W} \quad \left[\leftarrow \vec{a}^T \rightarrow \right] \begin{bmatrix} \uparrow & \uparrow \\ \vec{w}_1 & \vec{w}_2 \\ \downarrow & \downarrow \end{bmatrix}$$

1 by 2

$$\mathbf{Z} = \begin{bmatrix} \vec{a}^T \vec{w}_1 & \vec{a}^T \vec{w}_2 \end{bmatrix}$$

$(1 * 3) + (2 * 4)$
 $3 + 8$
 11

$(1 * 5) + (2 * 6)$
 $5 + 12$
 17

$$\mathbf{Z} = [11 \quad 17]$$

matrix matrix multiplication

$$A = \begin{bmatrix} 1 & -1 \\ 2 & -2 \end{bmatrix} \quad A^T = \begin{bmatrix} 1 & 2 \\ -1 & -2 \end{bmatrix} \quad W = \begin{bmatrix} 3 & 5 \\ 4 & 6 \end{bmatrix} \quad Z = A^T W = \begin{bmatrix} \leftarrow \vec{a}_1^T \rightarrow & \leftarrow \vec{a}_2^T \rightarrow \end{bmatrix} \rightarrow \begin{bmatrix} \uparrow \vec{w}_1 \downarrow & \uparrow \vec{w}_2 \downarrow \end{bmatrix}$$

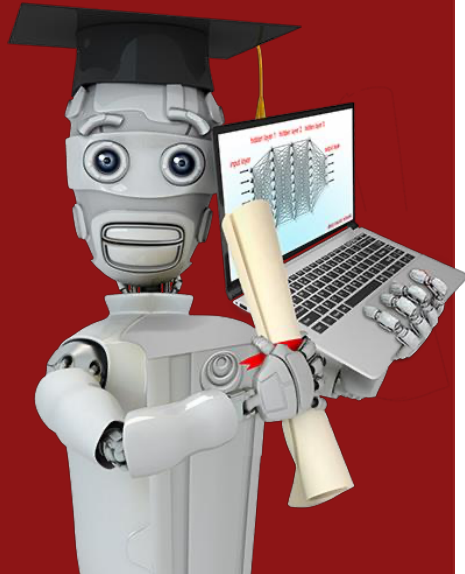
rows columns

$$\begin{array}{l} \text{row 1 col 1} \\ \text{row 2 col 1} \end{array} = \begin{bmatrix} \vec{a}_1^T \vec{w}_1 & \vec{a}_1^T \vec{w}_2 \\ \vec{a}_2^T \vec{w}_1 & \vec{a}_2^T \vec{w}_2 \end{bmatrix} \begin{array}{l} \text{row 1 col 2} \\ \text{row 2 col 2} \end{array}$$

$$\begin{array}{l} (-1 \times 3) + (-2 \times 4) \\ -3 + -8 \\ -11 \end{array} \quad \begin{array}{l} (-1 \times 5) + (-2 \times 6) \\ -5 + -12 \\ -17 \end{array}$$

$$= \begin{bmatrix} 11 & 17 \\ -11 & -17 \end{bmatrix}$$

general rules for
matrix multiplication
↪ next video!



Vectorization (optional)

Matrix multiplication rules

Matrix multiplication rules

$$\mathbf{A} = \begin{bmatrix} 1 & -1 & 0.1 \\ 2 & -2 & 0.2 \end{bmatrix} \quad \mathbf{A}^T = \begin{bmatrix} 1 & 2 \\ -1 & -2 \\ 0.1 & 0.2 \end{bmatrix} \quad \mathbf{W} = \begin{bmatrix} 3 & 5 & 7 & 9 \\ 4 & 6 & 8 & 0 \end{bmatrix} \quad \mathbf{Z} = \mathbf{A}^T \mathbf{W} = \begin{bmatrix} & & & \\ & & & \\ & & & \end{bmatrix}$$

The diagram illustrates the matrix multiplication rules for $\mathbf{Z} = \mathbf{A}^T \mathbf{W}$.

 - Matrix $\mathbf{A}$ is a 2x3 matrix with elements 1, -1, 0.1 in the first row and 2, -2, 0.2 in the second row.

 - Matrix $\mathbf{A}^T$ is a 3x2 matrix with elements 1, 2 in the first column, -1, -2 in the second column, and 0.1, 0.2 in the third column.

 - Matrix $\mathbf{W}$ is a 2x4 matrix with elements 3, 5, 7, 9 in the first row and 4, 6, 8, 0 in the second row.

 - Matrix $\mathbf{Z}$ is a 3x4 matrix, represented by a grid with 3 rows and 4 columns.

 The diagram also shows the row vectors $\vec{a}_1^T, \vec{a}_2^T, \vec{a}_3^T$ of $\mathbf{A}^T$ and the column vectors $\vec{w}_1, \vec{w}_2, \vec{w}_3, \vec{w}_4$ of $\mathbf{W}$.



Matrix multiplication rules

$$A = \begin{bmatrix} 1 & -1 & 0.1 \\ 2 & -2 & 0.2 \end{bmatrix} \quad A^T = \begin{bmatrix} 1 & 2 \\ -1 & -2 \\ 0.1 & 0.2 \end{bmatrix} \quad W = \begin{bmatrix} 3 & 5 & 7 & 9 \\ 4 & 6 & 8 & 0 \end{bmatrix} \quad Z = A^T W = \begin{bmatrix} \circ & & & \\ & \circ & & \\ & & \circ & \end{bmatrix}$$

3 by 4 matrix

$$\vec{a}_1^T \vec{w}_1 = (1 \times 3) + (2 \times 4) = 11$$

row 3 column 2

$$\vec{a}_3^T \vec{w}_2 = (0.1 \times 5) + (0.2 \times 6) = 1.7$$

0.5 + 1.2

row 2 column 3?

$$\vec{a}_2^T \vec{w}_3 = (-1 \times 7) + (-2 \times 8) = -23$$

-7 + -16

Matrix multiplication rules

$$A = \begin{bmatrix} 1 & -1 & 0.1 \\ 2 & -2 & 0.2 \end{bmatrix} \quad A^T = \begin{bmatrix} 1 & 2 \\ -1 & -2 \\ 0.1 & 0.2 \end{bmatrix} \quad W = \begin{bmatrix} 3 & 5 & 7 & 9 \\ 4 & 6 & 8 & 0 \end{bmatrix} \quad Z = A^T W = \begin{bmatrix} 11 & 17 & 23 & 9 \\ -11 & -17 & -23 & -9 \\ 1.1 & 1.7 & 2.3 & 0.9 \end{bmatrix}$$

$$\vec{a}_1^T \vec{w}_1 = (1 \times 3) + (2 \times 4) = 11$$

3 by 4 matrix

row 3 column 2

$$\vec{a}_3^T \vec{w}_2 = (0.1 \times 5) + (0.2 \times 6) = 1.7$$

0.5 + 1.2

row 2 column 3?

$$\vec{a}_2^T \vec{w}_3 = (-1 \times 7) + (-2 \times 8) = -23$$

-7 + -16

Matrix multiplication rules

$$A = \begin{bmatrix} 1 & -1 & 0.1 \\ 2 & -2 & 0.2 \end{bmatrix} \quad A^T = \begin{bmatrix} 1 & 2 \\ -1 & -2 \\ 0.1 & 0.2 \end{bmatrix} \quad W = \begin{bmatrix} 3 & 5 & 7 & 9 \\ 4 & 6 & 8 & 0 \end{bmatrix} \quad Z = A^T W = \begin{bmatrix} 11 & 17 & 23 & 9 \\ -11 & -17 & -23 & -9 \\ 1.1 & 1.7 & 2.3 & 0.9 \end{bmatrix}$$

3×2 2×4

can only take dot products
of vectors that are same length

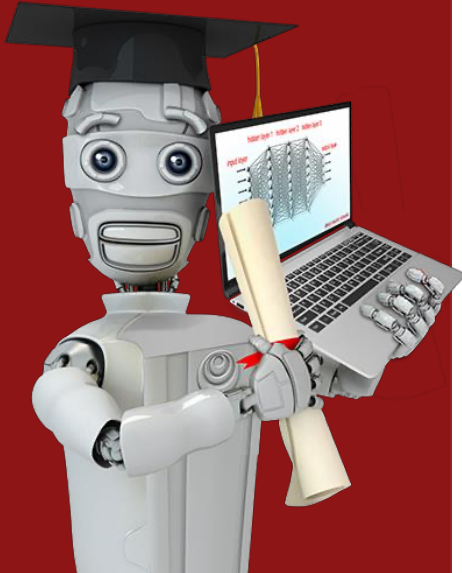
$[0.1 \ 0.2]$

length 2

$\begin{bmatrix} 5 \\ 6 \end{bmatrix}$

length 2

3 by 4 matrix
↳ same # rows as A^T
↳ same # columns as W



Vectorization (optional)

Matrix multiplication code

Matrix multiplication in NumPy

$$A = \begin{bmatrix} 1 & -1 & 0.1 \\ 2 & -2 & 0.2 \end{bmatrix} \quad A^T = \begin{bmatrix} 1 & 2 \\ -1 & -2 \\ 0.1 & 0.2 \end{bmatrix} \quad W = \begin{bmatrix} 3 & 5 & 7 & 9 \\ 4 & 6 & 8 & 0 \end{bmatrix} \quad Z = A^T W = \begin{bmatrix} 11 & 17 & 23 & 9 \\ -11 & -17 & -23 & -9 \\ 1.1 & 1.7 & 2.3 & 0.9 \end{bmatrix}$$

```
A=np.array([1,-1,0.1],  
            [2,-2,0.2]))
```

```
W=np.array([3,5,7,9],  
            [4,6,8,0]))
```

```
Z = np.matmul(AT,W)  
or Z = AT @ W
```

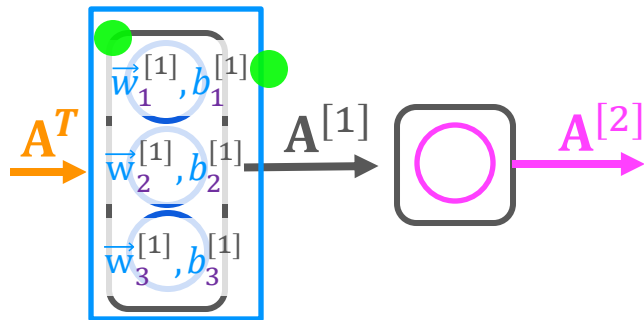
```
AT=np.array([1,2],  
             [-1,-2],  
             [0.1,0.2]))
```

```
AT=A.T  
      ↳ transpose
```

result

```
[[11,17,23,9],  
 [-11,-17,-23,-9],  
 [1.1,1.7,2.3,0.9]  
 ]
```

Dense layer vectorized



$$A^T = \begin{bmatrix} 200 & 17 \end{bmatrix}$$

$$W = \begin{bmatrix} 1 & -3 & 5 \\ -2 & 4 & -6 \end{bmatrix}$$

$$\vec{b} = \begin{bmatrix} -1 & 1 & 2 \end{bmatrix}$$

$$Z = A^T W + B$$

$$\begin{bmatrix} 165 & -531 & 900 \end{bmatrix}$$

$$z_1^{[1]} \quad z_2^{[1]} \quad z_3^{[1]}$$

$$A = g(Z)$$

$$\begin{bmatrix} 1 & 0 & 1 \end{bmatrix}$$

A

```
AT = np.array([[200, 17]])
```

```
W = np.array([[1, -3, 5],
              [-2, 4, -6]])
```

```
b = np.array([-1, 1, 2])
```

```
def dense(AT,W,b,g):
```

```
    z = np.matmul(AT,W) + b
```

```
    a_out = g(z)
```

```
    return a_out
```

```
[[1,0,1]]
```